

Early Stopping is Cheap

Fatima Elsheimy¹, Simon Holmgård Kamp², and Julian Loss²

¹ Yale University

² CISA Helmholtz Center for Information Security

Abstract. Minimizing both the round and communication complexity of Byzantine agreement (BA) is a fundamental question in distributed computing. A long line of works has focused on early-stopping deterministic protocols that can terminate within a number of synchronous rounds that is proportional to f , where f is the *actual number* of corruptions in an execution, as opposed to an upper bound t . This can lead to major benefits when $f \ll t$. A very different style of randomized protocol has focused on *player replaceable* BA protocols with communication complexity linear in the number of parties n and adaptive security, which rely on only a small and rotating subcommittee of parties to ever speak in the protocol. One downside of existing player-replaceable protocols is that they require $O(r)$ rounds to terminate with overwhelming probability $1 - 2^{-r}$. For applications demanding high security guarantees, this can easily become the bottleneck of a player replaceable protocol. Motivated by this gap in the literature, we give the first protocol that is simultaneously player-replaceable *and* early stopping (with overwhelming probability). Let $1 > \alpha > 0$ and $1 > \epsilon > 0$ be constants and let λ and κ denote suitable security parameters. Our protocol is secure against up to $t < (1 - \alpha) \cdot n/2$ adaptive Byzantine corruptions and terminates in $(1 + \epsilon) \cdot f$ many rounds with probability $1 - 2^{-\lambda}$, given $f \leq t$ corruptions. Moreover, our protocol has constant expected round complexity and communication bounded by $O(n \cdot \lambda^3 \cdot \kappa)$.

1 Introduction

Byzantine Agreement (BA) is a core problem in distributed computing. Given binary inputs, n parties aim to agree on a common output by running a protocol Π , despite up to $t < n/2$ *Byzantine* parties that may behave arbitrarily. The two key efficiency metrics of Byzantine agreement are (i) the *round complexity*: how many rounds does Π run in order to produce output and terminate for all non-Byzantine parties? (ii) the *communication complexity*: how many bits of communication do honest parties have to exchange in total in order to run Π ?

Dolev and Strong[11] and Dolev and Reischuk [9] show that deterministic protocols in the synchronous setting require at least $t + 1$ rounds and $O(n \cdot t)$ bits of communication, respectively. It is well-known that both of these lower bounds can be side-stepped by considering *randomized protocols*. In the randomized setting, the seminal works of Feldman and Micali [14] and Katz and Koo [21] demonstrate that expected $O(1)$ -round protocols are possible. Various

works, e.g., Abraham et al. [2], show that consensus with expected $o(n \cdot t)$ bits of communication is possible against an adversary that cannot adaptively corrupt parties and erase their previously sent messages from the network.

While these results demonstrate the power of randomized protocols, the lower bound of Karlin and Yao [20] (KY) shows that even a randomized BA protocol cannot guarantee correct termination in $O(\lambda)$ rounds with less than $O(\lambda^{-\lambda})$ error probability *for all executions* if the t corrupt parties make up a constant fraction of n . This lower bound was only recently matched by Ghinea et al. [19] and somewhat limits the utility and reliability of randomized protocols. Fortunately, the KY bound leaves a glimmer of hope: it does not account for executions in which the number of *actually corrupted parties* f is much lower than the upper bound t , i.e., $f \ll t$. A natural question is therefore whether it is possible to maintain some of the benefits of randomized protocols while simultaneously improving over the KY lower bound for executions where f is small. Answering this question positively, two prior works [30, 12] have shown randomized *early-stopping protocols* which terminate with all but negligible probability in $O(f)$ (rather than $O(t)$) rounds while simultaneously achieving $O(1)$ rounds in expectation.³ Unfortunately, both of these works are highly expensive in terms of communication. In this work, we give the first protocol which combines all of the following properties:

- It tolerates $(1 - \alpha) \cdot t/2$ corrupted parties, for some constant $\alpha > 0$.
- It terminates in $O(1)$ expected rounds.
- With high probability, it requires $O(\text{poly}(\lambda, \kappa) \cdot n)$ communication.
- It terminates with high probability in $(1 + \epsilon) \cdot f$ rounds, for an arbitrarily small constant $\epsilon > 0$. (We remark that in the cryptographic setting, this is the best achievable guarantee as perfect security is impossible.)

At a technical level, our protocol applies the (by now standard) *player-replaceability paradigm* to the recent early-stopping protocol of Elsheimy et al. [12]. Due to the unique challenges of early-stopping protocols, our work introduces several new ideas that may serve as a starting point for designing practical early-stopping protocols.

1.1 Technical Overview

Our starting point is the randomized Byzantine Agreement (BA) protocol of Elsheimy et al. [12]. At a high level, their early-stopping BA protocol (ELP24) proceeds in iterations, each lasting $d+9$ rounds, where d is a predefined constant. At the end of each iteration, the protocol guarantees one of two outcomes: either the honest parties reach agreement on a common output, or they collectively identify and exclude up to d additional malicious parties. Once no malicious parties are left to detect, the honest parties therefore agree and eventually are able to terminate from the protocol. It is easy to see that this happens at the

³ In fact, [30] is unauthenticated and achieves error-free termination in $\min\{f+2, t+1\}$ rounds for $t < n/8$, which is known to be optimal.

latest after $\lfloor f/d \rfloor + O(1) = f/d + O(1)$ iterations, which implies a total round complexity of $(d + 9) \cdot (f/d + O(1)) = f + 9f/d + O(d)$ rounds, which can be simplified to $(1 + \epsilon) \cdot f$, whenever $f = \Omega(1)$ and d is a constant. Their protocol incurs $O(n^4 \cdot f \cdot \kappa)$ communication.

Player-Replacable Consensus Protocols. To achieve subquadratic communication complexity, it is essential to eliminate all-to-all communication. We therefore adopt the player-replaceable consensus paradigm, first introduced by Micali et al. [6]. In this paradigm, different steps of the protocol are delegated to randomly selected committees, where each committee is responsible for executing a single step. Importantly, committee membership is secret until a party sends its one and only message, thus providing resilience against adaptive adversaries. Each party is independently elected to a committee with probability s/n , where s is a tunable parameter representing the expected committee size. In our setting, we set $s = \lambda$, where λ is a statistical security parameter. While committees are often elected using verifiable random functions (VRFs), we abstract away the implementation details and instead adopt the idealized committee-election functionality $\mathcal{F}_{\text{VRF}}$, as defined in [7] and formalized in fig. 7. This functionality models committee selection as a process that randomly and independently selects parties as needed for each round. Unlike some previous works such as [6, 22], our protocol – in line with Abraham et al. [2] – does not rely on memory-erasure assumptions, which require parties to erase their secret keys after sending a message. However, as with other subquadratic consensus protocols, our protocol relies on the no after-the-fact removal assumption, which circumvents a lower bound given in [2]. This assumption stipulates that the adversary cannot retroactively delete a message sent by a party P in round r before P became corrupt. Once P is corrupted, the adversary may inject additional round- r messages on behalf of P , but it cannot erase messages that P sent prior to corruption. Below, we provide a detailed explanation of each building block, along with our modifications that enable the integration of the approach by Elsheimy et al. with randomly sampled committees.

Correct-or-Detect Broadcast. The first building block of our protocol is the *Correct-or-Detect* (CoD) Broadcast protocol introduced by Fitzi and Nielsen [16]. This protocol, denoted $\Pi_{\text{FN-CoD}}$, builds upon the seminal broadcast protocol of Dolev and Strong, which runs in $t + 1$ rounds and guarantees correctness against any number of up to $t < n$ corrupted parties.

$\Pi_{\text{FN-CoD}}$ [16] is 1-biased: the designated sender sends their value to all parties if the value is 1, but refrains from sending anything otherwise. In all rounds $r \geq 1$, a party that receives a nested chain of r signatures on the value 1 from distinct signers that include the sender, it forwards this chain to all others, if r is the first round in which that party has received such a signature. Henceforth, we refer to such a chain of signatures simply as a *1-chain of length r* . The round in which a party first receives a 1-chain determines its output. For this reason: a crucial feature of $\Pi_{\text{FN-CoD}}$ is its *adjacency property*: if an honest party P receives

a 1-chain of length r in round r for the first time, then P can extend this chain by adding its own signature. This ensures that all honest parties receive a 1-chain of length $r + 1$ by round $r + 1$ at the latest (except if r is the last round of the protocol). This property underpins the protocol’s ability to achieve either correct broadcast or common detection of faults. Rather than achieving full broadcast, parties in $\Pi_{\text{FN-CoD}}$ terminate the protocol in two possible modes C (correct) and D (detect). If an honest party terminates in mode C , all parties agree on the sender’s value. If an honest party terminates in mode D , all honest parties detect a common set of at least d corrupted parties. As the protocol runs for a fixed $d + 4$ rounds, and each party sends at most one 1-chain of length at most d to every other party, its total communication complexity is $O(n^2 \cdot d \cdot \kappa) = O(n^2 \cdot \kappa)$ as d is constant.

A central challenge in player-replaceable consensus protocols is ensuring that committee members can convey their outputs to non-committee members. For instance, in the work of Chan et al. [5], which constructs a player-replaceable variant of the Dolev-Strong broadcast protocol, the standard technique of taking a majority vote over committee members’ outputs is not applicable due to their dishonest-majority setting. In our setting, although we assume an honest majority, we face a different—but related—challenge: we cannot naively rely on taking the majority output at the end of the protocol. The difficulty arises in preserving the adjacency property when combining random committee selection with the $\Pi_{\text{FN-CoD}}$ protocol. If an honest party *outside* the committee receives a valid 1-chain from a committee member in round r , it is not allowed to forward it and it cannot know if the committee member honestly sent the chain to all parties. Consequently, it cannot guarantee that all honest parties will receive the chain by round $r + 1$.

To address this issue, we modify the protocol such that the output for each sender is determined based on the first round r in which a threshold $\lambda/2$ of the elected committee has sent a valid chain for that sender, received by round r or earlier. By electing committees with an election probability that ensures having at least $\lambda/2$ honest parties and a strictly smaller number of corrupted parties (with overwhelming probability), we guarantee that at least one of the committee members forwarding the 1-chain is honest and hence will forward the 1-chain to all parties. This modification preserves adjacency and thus, guarantees the security properties of $\Pi_{\text{FN-CoD}}$ within a lightweight committee-based structure. Henceforth, we denote our committee-based CoD by $\Pi_{\text{d-CoD}}$.

Finally, note that our protocol naively overcomes the memory erasure assumption. This is because the $\Pi_{\text{d-CoD}}$ protocol is 1-biased (could be referred to as a unary action), meaning that parties can only send a 1-chain message. As a result, if a party sends a 1-chain and is subsequently corrupted, the adversary gains no advantage—since it can not send conflicting 0-chain message. The only subtlety to consider is that the output must be determined based on the first round where a chain was received from $\lambda/2$ committee members, as each committee member can be corrupted and made to send additional signature chains after it reveals itself. In terms of communication complexity, since communication occurs only

between the committee and all parties, the communication complexity of $\Pi_{\text{d-CoD}}$ is $O(\lambda \cdot n \cdot \kappa)$.

Banning Detected Parties. As in ELP24, we must ensure that parties identified as corrupt in $\Pi_{\text{d-CoD}}$ —i.e., those commonly detected by all honest parties—are prevented from participating in future protocol executions. To enforce this exclusion, ELP24 relies on a phase of gathering *proofs of participation (PoP)* prior to running $\Pi_{\text{FN-CoD}}$. This gathering protocol, termed Π_{PoP} , ensures that only parties that obtain a valid PoP can continue participating in the consensus protocol. Π_{PoP} and the ensuing exclusion mechanism are surprisingly simple. Each party P_i sends a signed message to party P_j if $P_j \notin \mathcal{F}_i$. Since honest parties do not include each other in their faulty lists, each honest party receives a PoP and is thus allowed to continue in the protocol. Conversely, parties that have commonly been identified as faulty do not receive PoPs and are permanently excluded. Subsequently, each party attaches its PoP to every message it sends during the protocol. Recipients accept a message from P_j only if it is accompanied by a valid PoP with respect to P_j . As there is all-to-all communication, the communication complexity is $O(n^2 \cdot \kappa)$.

Again, our protocol relies on a committee-based PoP version to reduce communication complexity. Specifically, before running Π_{PoP} , committees are elected and responsible for sending the PoP messages. To eliminate the need for memory-erasure assumptions, a distinct committee is elected for each recipient party, and thus, the best adversarial strategy is corrupting a random party. When checking whether it belongs to a committee, a party queries $\mathcal{F}_{\text{VRF}}$ using the index of the intended recipient. As a result, each party performs n such queries in total, one per potential recipient. Each committee member then sends a signature to the relevant parties who are not included in its faulty list. In terms of communication complexity, the protocol incurs $O(\text{poly}(\lambda) \cdot n \cdot \kappa)$. Moreover, since a valid PoP consists of at least $\lambda/2$ individual signatures—which cannot be aggregated using threshold signatures as the committees are ad-hoc and have no shared setup—each party must attach these signatures to its messages in all subsequent rounds.

(0, 1)-Graded Consensus from Correct-Or-Detect Broadcast. Following the approach of ELP24, we construct a $(0, 1)$ -graded consensus protocol using instances of the Correct-Or-Detect (CoD) broadcast. Each party P_i first uses CoD to broadcast its input. After all broadcasts complete, P_i observes the outputs of all CoD instances and determines both its output value and its grade accordingly. In the standard setting, if P_i observes $t + 1$ instances that terminate with value b and flag C (i.e., in correct mode), it outputs b with grade $g_i = 1$. Otherwise, for grade $g_i = 0$, it computes the majority bit over the outputs of all eligible parties—regardless of their termination mode.

To adapt this approach to our setting, we make two crucial modifications due to both communication constraints and the fact that we no longer run n parallel instances. First, we run a single instance of Π_{PoP} to obtain a valid PoP (proof

of participation), which parties use to participate in the subsequent rounds. In contrast, ELP24 runs PoP inside each instance of $\Pi_{\text{FN-CoD}}$. Since they run n such instances, this incurs n times the communication cost of $\Pi_{\text{FN-CoD}}$. Replicating this would lead to quadratic communication overhead in our setting, as our version of $\Pi_{\text{d-CoD}}$ already incurs $O(\text{poly}(\lambda) \cdot n \cdot \kappa)$ communication. Thus, we decouple the PoP generation into a standalone step to keep the communication overhead linear.

Second, we revise the output and grading logic to account for the fact that only expected λ instances of $\Pi_{\text{d-CoD}}$ are run, rather than n . Specifically, each party counts how many outputs with value 1 it receives, and uses the associated $\Pi_{\text{d-CoD}}$ flags to determine the grade. A party outputs value 1 if it at least $\lambda/2$ instances output one. This threshold ensures validity: assuming at least $\lambda/2$ honest parties in the committee, if all honest inputs are 1, then all honest parties will receive at least $\lambda/2$ outputs of 1 from the honest members. On the other hand, since there are fewer than $\lambda/2$ corrupted parties, if all honest parties input 0, no honest party can receive $\lambda/2$ ones. Ensuring validity for the *grade* requires that the output is $g_i = 1$ when honest parties are in pre-agreement. We achieve this by setting $g_i = 1$ if either (i) at least $\lambda/2$ outputs are 1 with flag C , or (ii) strictly more than $n - \lambda/2$ outputs are 0 with flag C . This choice ensures that when honest parties are in agreement, the grade reflects this.

To guarantee the consistency property—i.e., that all honest parties must output the same bit if a single honest party has grade 1—we rely on the CoD consistency guarantees. Suppose two honest parties output different bits and one of them has grade 1. Then, one must have seen at least $\lambda/2$ outputs of 1, and the other more than $n - \lambda/2$ outputs of 0. For the party who output grade 1, these outputs came with flag C . Since the thresholds sum to $n + 1$ they have to coincide on some party index. This would imply that the same message was received as two different bits two honest parties and one of them received flag C , violating the consistency property of $\Pi_{\text{d-CoD}}$ (as defined in definition 2). Finally, the protocol satisfies detection of commonly detected d -parties: if a party outputs with grade $g_i \neq 1$, then at least one $\Pi_{\text{d-CoD}}$ instance must have terminated with flag D .

Boosting $(0, 1)$ -Graded Consensus to $(0, 1, 2)$ -Graded Consensus. In the standard (folklore) approach to randomized Byzantine agreement with expected constant round complexity, we repeatedly invoke a $(0, 1, 2)$ -graded consensus protocol in each iteration. The output grade guides the protocol’s progress: if a party outputs with grade 2, it is confident that all honest parties agree on the same value and can safely terminate. If the output is grade 1, the party remains unsure—there is a possibility that some honest parties output with grade 0 or 2. In this case, the party keeps its current output value from $(0, 1, 2)$ -Graded consensus and continues to the next iteration. Finally, if the output is grade 0, the party is confident that there is no agreement and must update its value, typically by adopting the input of a some randomly elected leader or using a common coin. To adapt this, we construct a $(0, 1, 2)$ -graded consensus protocol. Our approach builds on the $(0, 1)$ -graded consensus described earlier, and augments it using *boosting rounds* to upgrade confidence levels. Specifically, after

running the $(0, 1)$ -graded consensus, each party participates in four additional rounds of a boosting protocol, following the design of $\Pi_{2\text{-GA}}$ from [26], using its output value from the initial $(0, 1)$ -graded consensus as input. In each boosting round, a fresh committee is elected to execute the boosting logic, as described previously, instead of standard all-to-all communication.

Finally, to eliminate the need for the memory erasure assumption, we tie a party's chances of being elected in the boosting rounds to the specific bit it is boosting, following the approach of [2, 5]. Specifically, when a party queries the $\mathcal{F}_{\text{VRF}}$ oracle to check its eligibility to send a boosting message, it must do so with respect to the bit it intends to boost. Without this bit-binding, an honest committee member could send a boosting message for one bit, be subsequently corrupted, and then be used by the adversary to send a conflicting boosting message for the opposite bit. Since our boosting rounds rely on collecting a quorum of more than $\lambda/2$ votes, such equivocation would allow the adversary to forge conflicting quorums and violate the security guarantees of the graded consensus protocol.

Common Coin As discussed earlier, when a party outputs a value with grade 0 from the $(0, 1, 2)$ -graded consensus protocol, it must pick a new random value to switch to. To do so, we implement such a protocol, denoted Π_{CC} , in a non-black-box manner. In Π_{CC} , each party is independently elected to a coin committee with probability λ/n . If a party P_i is elected, it broadcasts its VRF string output to all other parties. Upon collecting the VRF outputs sent by the elected committee members, each party computes the output coin value c_i by taking the minimal VRF output among them and outputs the least significant bit of its hash. This construction yields a good coin—i.e., one that is both common (identical for all honest parties) and indistinguishable from random before the protocol starts—under the condition that the least VRF output belongs to a party who was honest at the time of sending it. We later prove that we get such a coin with probability $1/2$.

From $(1 + \epsilon)f$ to $f + O(\sqrt{f})$ rounds. Elsheimy et al., [13] achieves a round complexity of $f + O(\sqrt{f})$ by incrementally increasing the parameter d in $\Pi_{\text{FN-CoD}}$ by 1 in each successive iteration, starting with $d = 1$ in the first iteration. Since it uses CoD in a black box fashion, the same technique is applicable in our setting. However, to keep the presentation focused we use a fixed $d = c$ for constant c . For full analysis, we refer the reader to [13].

1.2 Related Work

Early stopping. Byzantine Agreement has been a central topic in distributed computing since its introduction in the seminal work of Shostak, Pease, and Lamport [23]. An important advancement was made by Dolev and Strong [11], who showed that in synchronous networks, $t + 1$ rounds are both necessary and sufficient for agreement when up to $t < n/2$ parties may be faulty (and up to $t < n$ in the case of broadcast). Dolev et al. [10] initiated the study of early stopping in Byzantine Agreement, focusing on executions where only $f < t$ parties

are actually corrupted. They refined the Dolev-Strong lower bound by proving that at least $\min(f + 2, t + 1)$ rounds are necessary in such cases. This bound has since been shown to be tight in various settings. In the information-theoretic regime, Berman et al. [4] first presented an exponential-time protocol with optimal resilience for up to $t < n/3$ faults. Garay and Moses later proposed a polynomial-communication protocol [18, 17], though it incurred a slightly suboptimal round complexity of $\min(f + 5, t + 1)$. Abraham and Dolev [3] later resolved this, presenting the first protocol with polynomial communication, optimal resilience, and optimal round complexity of $\min(f + 2, t + 1)$. In the authenticated setting for $t < n/2$ Perry and Toueg [28] provided a BA protocol with early stopping in $2 \cdot \min(t + 1, f + 2)$ rounds. Recently Elsheimy et al. [12] improved the early stopping condition to $(1 + \epsilon)f$ rounds and later refined the techniques to obtain $f + 6\sqrt{f} + 6$ rounds. They also achieved a bound of $\min(f + 3, t + 1)$ rounds, though at the cost of exponential communication [13]. In the dishonest majority setting the recent work of Loss and Nielsen [24] constructs broadcast in $O(\min(t, f^2))$ rounds for $t < n$ and Deligios et al. [8] cleverly combined their techniques with insights from Wan et al. [29] to obtain asymptotically optimal $O(\min(t, f))$ rounds for $t < (1 - \epsilon)n$ where $\epsilon > 0$ is constant.

Randomized Protocols. An alternative way around the Dolev-Strong lower bound is to rely on randomization. In this realm, Karlin and Yao [20] showed that BA cannot be solved in $O(\lambda)$ rounds with less than $O(\lambda^{-\lambda})$ error probability. Ghinea et al. [19] matched this lower bound by constructing a synchronous BA protocol tolerating $t < (1 - \epsilon)n/2$ corruptions and terminating in $O(\lambda/(\log \lambda))$ for constant ϵ with negligible error in λ . While the Ghinea et al. protocol runs for a fixed number of rounds before terminating with asymptotically optimal error probability, it does not allow earlier termination in any execution. In contrast, a wide variety of protocols achieve expected constant round complexity and still terminate in $O(\lambda)$ rounds with negligible error in λ : with $t < n/3$ and information theoretic security [15] or $t < n/2$ and authentication [21]. The combination of early stopping with expected constant rounds termination has previously been studied by Zamsky [30] who provides an error-free solution but with $t < n/8$ and Elsheimy et al. [12] which we draw on for our communication-efficient construction.

Communication complexity. Dolev and Reischuk [9] showed that broadcast cannot be solved deterministically with $o(n^2)$ communication with a linear number of adaptive corruptions and Abraham et al. [1] generalized this to randomized broadcast protocols. As BA implies broadcast in synchrony for $t < n/2$, this implies that BA cannot be solved with $o(n^2)$ communication under strong adaptive corruption of $\Theta(n)$ parties. On the positive side several works [6, 22, 2] have provided subquadratic constructions by relaxing the adversary's adaptive capabilities.

1.3 Paper Organization

Section 2 introduces the necessary definitions and outlines our network model. Sections 3, 4, 5, and 6 present the core building blocks of our construction, along with their corresponding correctness proofs, including the participation protocol, the correct-or-detect broadcast protocol, and graded consensus protocols. Finally, in Section 7, we describe our Byzantine Agreement protocol, and analyze its correctness, communication complexity, and round complexity.

2 Preliminaries

We begin by introducing the model as well as basic definitions.

Network and Setup Assumptions. We denote κ as the cryptographic security parameter and λ as the statistical security parameter. Without loss of generality, we assume $\lambda \leq \kappa$. We assume a fully connected network consisting of n parties $\{P_1, \dots, P_n\} = \mathcal{P}$, where each pair of parties shares an authenticated communication channel. We consider a *synchronous network model*, meaning all parties have access to a synchronized clock and there exists a known upper bound Δ on the delivery time of messages between honest parties. Consequently, protocols can proceed in synchronized rounds of length Δ , such that any message sent by an honest party at the start of a round is guaranteed to be delivered to all honest parties by the end of that round.

We further assume the existence of a public key infrastructure (PKI) supporting a digital signature scheme with efficient signing and verification algorithms, denoted **Sign** and **Verify**, respectively. Each party P_i holds a public key pk_i known to all other parties, and a corresponding private key sk_i known only to P_i . A party P_i can generate a signature on message m as $\text{sig}(m) := \text{Sign}(\text{sk}_i, m)$, which can be verified using $\text{Verify}(\text{pk}_i, \text{sig}(m), m)$. We say a signature $\text{sig}(m)$ is *valid* if $\text{Verify}(\text{pk}_i, \text{sig}(m), m) = 1$. For notational convenience, we use the shorthand $\langle m \rangle_i$ to denote the signed message tuple $(m, \text{sig}(m))$ produced by party P_i . We treat signatures as having perfect unforgeability and report their size as κ bits when analyzing communication complexity.

Adversary Model. We consider an adaptive, computationally-bounded Byzantine adversary that can corrupt up to $t < (1 - \alpha) \cdot n/2$, for $0 < \alpha < 1$, parties at any point during a protocol execution. A corrupt (or malicious) party P_i is under full control of the adversary and may deviate arbitrarily from the protocol. In particular, the adversary learns P_i 's signing key sk_i , which allows it to sign messages on P_i 's behalf. Since no erasure is assumed, this includes messages from rounds where P_i has already sent something. As a special case, this also allows the adversary to inject new messages in round r on behalf of a newly corrupted party in round r . However, we do not allow the adversary to delete any undelivered messages of a newly corrupted party P_i that P_i sent while it was still honest.

Committee Election In our protocols, parties are elected to committees that each perform a single task which we will call a *role*. To elect the committee for each role we follow the common design pattern [6,7,2] of evaluating a *verifiable random function* (VRF) [25] on a string which uniquely identifies the role and then based on the output decide eligibility deterministically in a way that on average filters out the desired number of parties. Concretely, we will use the ideal functionality $\mathcal{F}_{\text{VRF}}$ defined by David et al. [7] to model VRFs where the output can be considered a uniform random string sampled at query time. The functionality additionally outputs a proof, which other parties can use to verify the output. We defer the reader to [7] for a secure implementation in the random oracle model but include a description of the functionality in fig. 7. The functionality allows parties to register keys at any point during the protocol and provides security against maliciously generated keys. But this is beyond the scope of our study. We will simply assume that each honest party P_i has registered to the functionality using the KeyGen command before the first round of the protocol to obtain a key vk_i , and that for each (potentially corrupted) party P_i : a verification key vk_i is given as part of the PKI which for honest parties will be the verification key received from the functionality.

All committees will have expected size λ , which in line with David et al. is implemented by setting a difficulty level $D = \frac{\lambda}{n} \cdot 2^{\ell_{\text{VRF}}}$ (where ℓ_{VRF} is the number of bits in the VRF output) and letting each party be eligible to speak for the role identified by m whenever their VRF on m evaluates to an output $y_{\text{VRF}} \in \{0, 1\}^{\ell_{\text{VRF}}}$ for which $y_{\text{VRF}} < D$. When describing a protocol from the view of a party P_i we will use the notation

- $(b_i, y_{\text{VRF}_i}, \pi_i) = \text{Elect}_{\text{VRF}}(m)$ to mean that P_i sends $(\text{EvalProve}, m)$ to $\mathcal{F}_{\text{VRF}}$ and upon receiving output $(\text{Evaluated}, y_{\text{VRF}}, \pi)$ lets $b_i = 1$ if $y_{\text{VRF}} < D$ and otherwise $b_i = 0$. Then lets $y_{\text{VRF}_i} = y_{\text{VRF}}$ and $\pi_i = \pi$.
- $b_j = \text{Verify}_{\text{VRF}}(m, y_{\text{VRF}}, \pi, j)$ to mean that P_i sends $(\text{Verify}, m, y_{\text{VRF}}, \pi, \text{vk}_j)$ to $\mathcal{F}_{\text{VRF}}$ and upon receiving output $(\text{Verified}, m, y_{\text{VRF}}, \pi, f)$: lets $b_j = 1$ iff $f = 1$ and $y_{\text{VRF}} < D$.

In the interest of readability, we omit the session identifier *sid* which is implicitly defined by the context. We let both ℓ_{VRF} and the size of a proof π be linear in the computational security parameter κ .

The following lemma shows that in the resulting committees: $\lambda/2$ is a lower bound on honest parties and upper bound on corrupted parties, except with negligible probability in λ assuming that at most $t < (1 - \alpha)n/2$ parties are adaptively corrupted independently of the their individual queries to $\mathcal{F}_{\text{VRF}}$.

Lemma 1 (Sampling honest majority committees). *Assuming the adversary corrupts up to $t < (1 - \alpha)n/2$ parties independently of honest calls to $\text{Elect}_{\text{VRF}}(\mathbf{m})$: the number of honest parties for which $(1, \cdot, \cdot) = \text{Elect}_{\text{VRF}}(\mathbf{m})$ is at least $\lambda/2$ and the number of corrupted parties for which $(1, \cdot, \cdot) = \text{Elect}_{\text{VRF}}(\mathbf{m})$ is strictly less than $\lambda/2$ except with negligible probability in λ .*

Proof. In any k evaluations of $(b, \cdot, \cdot) = \text{Elect}_{\text{VRF}}(\mathbf{m})$ made by different parties, the bits returned are independent Bernoulli random variables each with success

probability λ/n and an expected sum of $k \cdot \lambda/n$. We can thus apply a Chernoff bound to say that the probability that the sum deviates from its expected value by some constant fraction vanishes exponentially in the expected size of the committee: λ .

Definition 1 (Byzantine Agreement). *Let Π be protocol executed among parties $P_1, \dots, P_n$, where each party P_i holds an input $v_i \in \{0, 1\}$ and outputs a value $y_i \in \{0, 1\}$ upon terminating. A protocol Π achieves Byzantine Agreement, if the following properties hold whenever at most t parties are corrupted.*

- *Validity: If every honest party P_i inputs $v_i = v$, then all honest parties output $y_i = v$;*
- *Consistency: All honest parties output the same value v .*
- *Termination: Every honest party terminates.*

Next, we define a role-based Parallel Correct or Detect Broadcast primitive. It is important to note that our definition differs from the Correct or Detect Broadcast in both [16] and [12]. As in [12], each honest party P_i inputs a faulty list $\mathcal{F}_i$ along with its initial value v_i . Essentially, malicious parties included in the initial faulty list of all honest parties are excluded from participating in the protocol. Additionally, malicious parties identified during the protocol execution are added to the party's initial faulty list $\mathcal{F}_i$. The parties then return the updated faulty list $\mathcal{F}_i^*$, with the notation $\mathcal{F}_i^*$ used to distinguish it from the initial input faulty list $\mathcal{F}_i$. Beyond performing a straightforward translation to parallel CoD – by giving each party an input and outputting a value and correctness flag for each other party – we additionally impose restrictions on the validity property that depend on whether the sender was elected to speak for the role associated with the CoD instance: While an honest party who is elected to speak for the role will have the usual validity guarantees of an honest designated sender in CoD, the value sent by any party (Byzantine or not) who is not elected to speak should always be 0 and the flag C . This will be crucial when using the protocol to construct graded agreement with detection efficiently based on sampling small committees.

Definition 2 (Parallel Correct or Detect Broadcast (d -PCoD)). *Let Π be protocol executed by parties $P_1, \dots, P_n$. It is defined in relation to the functionality $\mathcal{F}_{\text{VRF}}$ and parameterized by a role role and phase ρ where each party P_i holds input $v_i \in \{0, 1\}$. In addition, each party inputs a list of faulty parties $\mathcal{F}_i \subset \mathcal{P}$ and outputs n values $y_i^1, \dots, y_i^n \in \{0, 1\}^n$, n corresponding flags $\text{det}_i^1, \dots, \text{det}_i^n \in \{C, D\}^n$, and an updated faulty list $\mathcal{F}_i^* \subset \mathcal{P}$ upon terminating. Π achieves Correct or Detect Broadcast (CoD), if the following properties hold whenever at most t parties are corrupted and no honest party is in the faulty list of another honest party.*

- *$\mathcal{F}$ -soundness: If an honest party P_i outputs $\mathcal{F}_i^*$, then $\mathcal{F}_i^*$ consists only of corrupted parties. Furthermore, $\mathcal{F}_i \subseteq \mathcal{F}_i^*$.*
- *Consistency: If $\text{det}_i^k = C$ for some honest party P_i , then every honest party P_j outputs $y_j^k = y_i^k$.*

- *Validity 1:* If P_i is honest, has input v_i , and $\text{Elect}_{\text{VRF}}(\text{role}||\rho)$ returns $(1, \cdot, \cdot)$ when queried by P_i : every honest party P_j outputs $y_i^j = v$ $\det_i^j = C$.
- *Validity 2:* If some honest party P_i outputs $y_i^j = 1$ or $\det_i^j = D$, then some honest party previously computed $\text{Verify}_{\text{VRF}}(\text{role}||\rho, \cdot, \cdot, j) = 1$.
- *d-Detection:* If for some honest party P_i , $\det_i^k = D$ for any party P_k , then an additional d parties are added to the faulty lists of all honest parties; that is,

$$\left| \left(\bigcap_{P_j \in \mathcal{H}} \mathcal{F}_j^* \right) \setminus \left(\bigcap_{P_j \in \mathcal{H}} \mathcal{F}_j \right) \right| \geq d.$$
- *Termination:* Every honest party terminates.

As in [12] we will be using CoD to construct graded agreement protocols with a *detection* property saying that if less than d new parties are commonly detected as faulty, then all parties received an output with the maximum grade.

Definition 3 ((0, 1)-Graded d -Detecting Agreement). Let Π be a protocol executed by parties $P_1, \dots, P_n$, where each party P_i inputs $v_i \in \{0, 1\}$ and a list of faulty parties $\mathcal{F}_i \subset \mathcal{P}$ and outputs a value $y_i \in \{0, 1\}$, a grade $g_i \in \{0, 1\}$, and an updated faulty list $\mathcal{F}_i^* \subset \mathcal{P}$ upon terminating. Π achieves (0, 1)-Graded d -Detecting Agreement if the following properties hold whenever at most t parties are corrupted and for all honest parties P_i , $\mathcal{F}_i$ contains only corrupted parties.

- *Graded Validity:* If all honest parties P_i have the same input value $v_i = v$ then all honest parties output $y_i = v$ and $g_i = 1$.
- *Graded Consistency:* If an honest party P_i outputs $g_i = 1$ and y_i , then for every honest party P_j who outputs y_j : $y_i = y_j$.
- *d-Detection:* If two honest parties P_i and P_j output $y_i = 1$ and $y_j = 0$, respectively, then an additional d parties are added to the faulty lists of all honest parties; that is, $\left| \left(\bigcap_{P_j \in \mathcal{H}} \mathcal{F}_j^* \right) \setminus \left(\bigcap_{P_j \in \mathcal{H}} \mathcal{F}_j \right) \right| \geq d$.
- *Soundness:* If an honest party P_i outputs $\mathcal{F}_i^*$, then $\mathcal{F}_i^*$ consists only of corrupted parties. Furthermore, $\mathcal{F}_i \subseteq \mathcal{F}_i^*$.
- *Termination:* Every honest party terminates.

Definition 4 ((0, 1, 2)-Graded d -Detecting Agreement). Let Π be a protocol executed by parties $P_1, \dots, P_n$, where each party P_i inputs $v_i \in \{0, 1\}$ and a list of faulty parties $\mathcal{F}_i \subset \mathcal{P}$ and outputs a value $y_i \in \{0, 1, \perp\}$, a grade $g_i \in \{0, 1, 2\}$, and an updated faulty list $\mathcal{F}_i^* \subset \mathcal{P}$ upon terminating. A protocol Π achieves (0, 1, 2)-Graded d -Detecting Agreement if the following properties hold whenever at most t parties are corrupted and for all honest parties P_i , $\mathcal{F}_i$ contains only corrupted parties.

- *Graded Validity:* If all honest parties P_i have the same input value $v_i = v$ then all honest parties output $y_i = v$ and $g_i = 2$.
- *Graded Consistency:* Let P_i and P_j denote honest parties that output y_i, g_i and y_j, g_j , respectively. Then (1) $|g_i - g_j| \leq 1$ and (2) $g_i, g_j \geq 1$ implies that $y_i = y_j$.

- *d-Detection*: If any honest party P_i outputs $g_i < 2$, then an additional d parties are added to the faulty lists of all honest parties; that is,

$$\left| \left(\bigcap_{P_j \in \mathcal{H}} \mathcal{F}_j^* \right) \setminus \left(\bigcap_{P_j \in \mathcal{H}} \mathcal{F}_j \right) \right| \geq d.$$
- *Soundness*: If an honest party P_i outputs $\mathcal{F}_i^*$, then $\mathcal{F}_i^*$ consists only of corrupted parties. Furthermore, $\mathcal{F}_i \subseteq \mathcal{F}_i^*$.
- *Termination*: Every honest party terminates.

3 Proof of Participation (Π_{PoP})

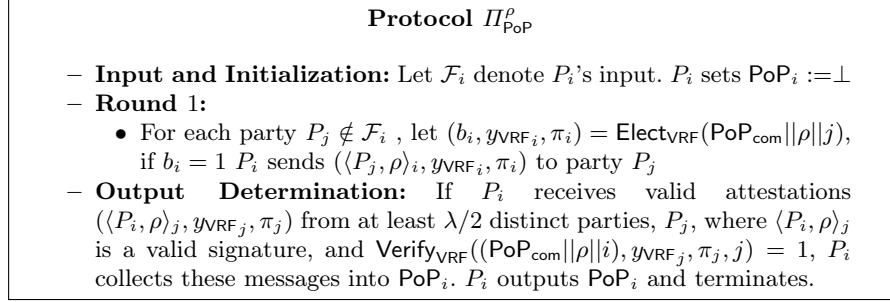
A Proof of Participation is simply a set of signatures provided by honest parties to other parties who they have not yet detected as faulty.

Definition 5 (Proof of Participation). A proof of participation PoP_i for a party $P_i \in \mathcal{P}$ is a collection of $\lambda/2$ triples of signatures, VRF outputs, and proofs of the form $(\langle P_i, \rho \rangle_{j_l}, y_{\text{VRF}_j}, \pi_j)$ from distinct parties $P_{j_1}, \dots, P_{j_{t+1}} \in \mathcal{P}$. We say that PoP_i is valid if for all $l \in [\lambda/2]$, $\langle P_i, \rho \rangle_{j_l}$ is valid with respect to pk_{j_l} and $\text{Verify}_{\text{VRF}}((\text{PoP}_{\text{com}} || \rho || i), y_{\text{VRF}_j}, \pi_j, j) = 1$.

At a high level, the Proof of Participation protocol, denoted Π_{PoP}^ρ , enables each party to obtain a proof PoP attesting to its honesty in order for it to be allowed to participate in phase ρ . Our design follows a similar logic to that of the protocol in [16]. In [16], the protocol proceeds in a single round: each party P_i sends a message $\langle P_j, \rho \rangle_i$ to every party $P_j \notin \mathcal{F}_i$, thereby asserting the honesty of P_j . If a party P_j receives at least $t + 1$ such messages, it uses them to construct its PoP proof. However, this naive approach incurs a quadratic communication overhead. To achieve subquadratic communication, we modify the design by electing a committee, of expected size λ , which is responsible for sending such honesty attestations to each party. Note that while this amounts to n separate committees each of expected size λ , the communication complexity remains $O(n\lambda\kappa)$ since each committee only speaks to the parties they are providing a PoP to. The motivation for having separate PoP committees for each party is that it avoids reliance on secure erasure. As a strawman consider having a single PoP committee: then whenever a party attests to the honesty of another party and thus reveals itself to be on a PoP committee that is responsible for providing PoPs to all parties, then it can be corrupted and used to fabricate PoPs for all corrupted parties. But if it only reveals itself as being on the PoP committee for a party P_i when it is already willing to attest to the honesty of P_i , then it is a priori of no more use in fabricating PoPs for some $P_j \neq P_i$ than any other party, since queries to $\mathcal{F}_{\text{VRF}}$ on different strings are sampled independently. Under this scheme, a PoP proof is considered valid if it consists of at least $\lambda/2$ honesty messages from the members that were elected on the committee for the receiver of the PoP , which we formally prove below.

Next, we define the two primary properties of Π_{PoP}^ρ in Lemma 2 and Lemma 3.

Lemma 2. Assume no honest party P_j is in the faulty list $\mathcal{F}_i$ of any other honest party P_i . Then, each honest party P_j outputs a valid PoP_j .

**Fig. 1.** Code of Π_{PoP}^ρ for party P_i .

Proof. By lemma 1, there are at least $\lambda/2$ honest parties on the committee of each party. Each honest party P_i for which $\text{Elect}_{\text{VRF}}(\text{PoP}_{\text{com}} || \rho || j) = (1, y_{\text{VRF}_i}, \pi_i)$ sends $(\langle P_j, \rho \rangle_i, y_{\text{VRF}_i}, \pi_i)$ to party P_j unless $P_j \in \mathcal{F}_i$ which by assumption is not the case for honest P_j . Thus, every honest party P_i will receive at least $\lambda/2$ messages of the form $\langle P_i, \rho \rangle_j$ from its PoP committee. Consequently, every honest party sets its output PoP_i to those received messages. $\square$

Lemma 3. Assume there exists some party P_j such that $P_j \in \mathcal{F}_i$ for all honest parties $P_i \in \mathcal{P}$ by the beginning of round 1. Then, P_j does not output a valid PoP_j .

Proof. From lemma 1, there are less than $\lambda/2$ malicious parties in any PoP committee. No honest party will send $(\langle P_j, \rho \rangle_i, y_{\text{VRF}_i}, \pi_i)$ to $P_j \in \mathcal{F}_i$. Thus, P_j can collect at most $t < \lambda/2$ such messages, which are not enough to form PoP_j . $\square$

Communication Complexity. The combined communication complexity of Π_{PoP}^ρ is $O(\lambda \cdot n \cdot \kappa)$ as $O(\lambda)$ parties are sending a signature to each of the n parties. The size of each PoP is $O(\lambda \cdot \kappa)$.

4 Correct or Detect Broadcast Protocols ($\Pi_{\text{d-CoD}}$)

We present a parallel committee based version of the Correct-or-Detect broadcast primitive presented by Fitzi and Nielsen. In this version, each party may act as a sender but can only speak if they are elected to be on the committee associated with the instance. This committee is uniquely defined by the role and iteration parameters role and ρ and we use this string as input to the leader election. Similarly, to avoid quadratic communication, each other party is only allowed to forward signature chains if they are elected to be on a subcommittee of an elected sender. Each party could be elected for the role $\text{role} || \rho$ and would then send its input bit following the CoD logic of Fitzi and Nielsen. Since each

sender may act at different times, the subcommittees need to be sampled independently, which we achieve by querying $\mathcal{F}_{\text{VRF}}$ on the string $\text{role}||\rho||j$, where j is the index of the sender.

Whereas Elsheimy et al. augmented the signature chains of Fitzi and Nielsen to include valid PoPs for each party, we now additionally require the designated sender to be elected for the role $\text{role}||\rho$ and that each subsequent signer was elected for the role of the subcommittee $\text{role}||\rho$. Additionally, since each party might be a sender, we say that a chain is a valid signature chain for P_i if P_i signed the first message of the chain.

Definition 6 (Signature Chain). *Let $m \in \{0, 1\}^*$, let $k \in \mathbb{N}$, and let PoP_k be the proof of participation of party P_k as per Definition 5. We write $\langle m \rangle_\sigma$ to denote the nested messages and signatures $\langle \dots \langle \langle m, \text{PoP}_{j_1}, y_{\text{VRF}_{j_1}}, \pi_{j_1} \rangle_{j_1}, \text{PoP}_{j_2}, y_{\text{VRF}_{j_2}}, \pi_{j_2} \rangle_{j_2}, \dots, \text{PoP}_{j_k}, y_{\text{VRF}_{j_k}}, \pi_{j_k} \rangle_{j_k}$, where $j_1, \dots, j_k$ are distinct values in $[n]$, and refer to σ as a signature chain of length k for P_{j_1} . The expression $\langle m \rangle_\sigma$ is said to be valid for P_{j_1} if, $\text{Verify}_{\text{VRF}}(\text{role}||\rho, y_{\text{VRF}_{j_1}}, \pi_{j_1}, j_1) = 1$, and for $l \in [2, k]$: $\text{Verify}_{\text{VRF}}(\text{role}||\rho||j_1, y_{\text{VRF}_{j_l}}, \pi_{j_l}, j_l) = 1$, for $l \in [1, k]$: the signature with respect to pk_{j_l} is valid, and the proof of participation PoP_{j_l} is valid.*

In the CoD protocol of Fitzi and Nielsen, the output bit and flags are determined by the first round in which each party receives a valid chain. The central observation is that if an honest party receives a valid chain (in their case for a designated sender) in round r , then all honest parties receive a valid chain at the latest in round $r + 1$. While this remains true in our protocols for the honest *committee members* with respect to each sender, it would not be sound for parties who are not on the committee to determine each output based on the first round in which they receive a valid chain from a sender. This is because (in order to reduce communication complexity) they are not instructed to forward chains unless they are in the subcommittee of the relevant sender, and thus they cannot conclude that the remaining honest parties will receive a valid chain in the following round. Instead, we will determine the output for each sender based on the first round in which the majority of the committee has sent a valid chain for that sender.

Regarding security without secure erasure: it is worth noting that a sender who is on the committee is free to send either 0 or 1, which seems to break with the erasure-free design pattern of Abraham et al. But it remains sound because an honest sender who is sending 0 can only do so by inaction. Since it never speaks it does not reveal itself and cannot be targeted for corruption based on the knowledge that it is on the committee. On the other hand, the only valid action an honest sender may take is to send a 1 chain in case it wants to send the bit 1. Once it has sent a valid chain, it cannot be clawed back by the adversary. The same logic applies to the subcommittees who forward the chains: if they do not forward a chain, then they do not reveal themselves. If they do forward a chain, then corrupting them only gives the ability to perform the same action in a later round. But since the protocol only cares about the first round in which $\lambda/2$ valid chains from distinct parties are received: sending another chain in a later round has no effect.

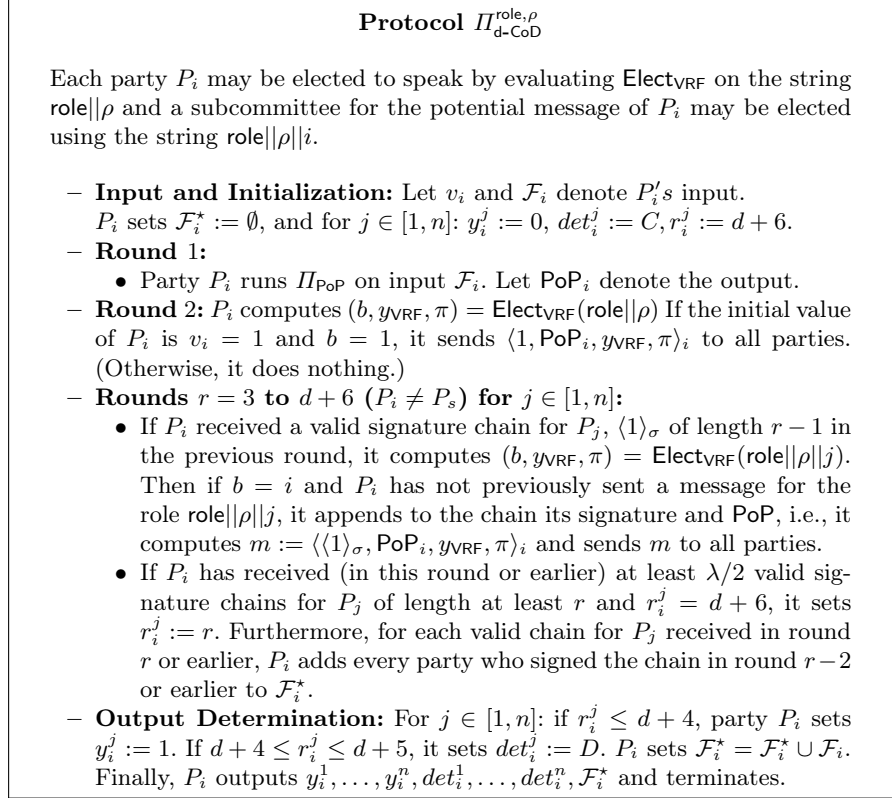


Fig. 2. Code of $\Pi_{\text{d-CoD}}$ for party P_i .

For all of the following arguments we assume that no honest parties are in honest faulty lists and that at most $t < (1 - \alpha)n/2$ parties are corrupted.

We first show that when an honest party *who is chosen to speak* sends a valid chain for some party P_j , then ever honest P_i determine their output for P_j , y_i^j , in the following round (if not earlier).

Lemma 4. *If some honest party sends a valid signature chain for P_j in round r , then for any honest party P_i : $r_i^j \leq r + 1$ at the time of output.*

Proof. First note that if an honest party sends a signature chain for P_j in round r , then it received $(1, \cdot, \cdot)$ on its query to $\text{Elect}_{\text{VRF}}(\text{role}||\rho||j)$ or it is the sender P_j who received $(1, \cdot, \cdot)$ on its query to $\text{Elect}_{\text{VRF}}(\text{role}||\rho)$. If $r = d + 6$ there is nothing to show, as this is the default value of r_i^j and it may only be set to a lower value during the protocol. Otherwise, all honest committee members will received the chain in round r and extend and forward the signature chain in round $r + 1$, unless they previously sent a chain. Lemma 1 shows that there are at least $\lambda/2$ honest committee members. So all parties will receive $\lambda/2$ valid chains, no later than round $r + 1$ and set their r_i^j value to $r + 1$, unless they previously set it to lower value. $\square$

On the other hand: whenever an honest party updates its r_i^j value for sender P_j , then at least one honest committee member must have sent a signature chain for P_j .

Lemma 5. *If some honest party sets $r_i^j = r \leq d + 5$, then an honest party sent a valid signature chain for P_j in round r or earlier.*

Proof. Assume some honest party P_i sets $r_i^j = r \leq d + 5$. This means that P_i has received valid signature chains for P_j from at least $\lambda/2$ parties on the committee in round r_i^j . By lemma 1 this must include a chain from an honest party. $\square$

It follows that the invariant that honest parties determine their output (for the same sender) in adjacent rounds from the protocol of Fitzi and Nielsen is maintained.

Lemma 6. *For any two honest parties P_i and P_j , $|r_i^k - r_j^k| \leq 1$.*

Proof. Assume WLOG that $r_i^k \leq r_j^k$. If $r_i^k = d + 6$ there is nothing to show, as this would imply $r_i^k = r_j^k$. So, we only need to consider executions where $r_i^k \leq d + 5$. Now lemma 5 says that P_i has received a valid signature chain for P_k from an honest party in round r_i^k and it follows from lemma 4 that for all honest P_j : $r_j^k \leq r_i^k + 1$. In conclusion $|r_i^k - r_j^k| \leq 1$. $\square$

This implies consistency following the same argument as in Fitzi and Nielsen.

Lemma 7 (Consistency). *If $\text{det}_i^k = C$ for some honest party P_i , then every honest party P_j outputs $y_j^k = y_i^k$.*

Proof. If P_i outputs flag $\text{det}_i^k = C$, then by construction of the output determination rule $|r_i^k - r_j^k|$ would need to be at least 2 for P_j to output $y_j^k \neq y_i^k$. But this cannot happen by lemma 6. $\square$

Lemma 4 also implies validity 1.

Lemma 8 (Validity 1). *If P_i is honest, has input v_i , and $\text{Elect}_{\text{VRF}}(\text{role}||\rho)$ returns $(1, \cdot, \cdot)$ when queried by P_i : every honest party P_j outputs $y_i^j = v$ $\text{det}_j^i = C$.*

Proof. If the sender P_i is honest, then: a) If it receives input 0, no valid signature chains for P_i can exist, so all parties output 0. b) If it receives input 1 and is elected to speak, it will send a signature chain in round 1, and (cf. lemma 4 every honest party P_i sets $r_i = 2$, which results in outputting $y_i = 1$. $\square$

Validity 2 holds because a sender who is not elected cannot form a valid signature chain for itself.

Lemma 9 (Validity 2). *If some honest party P_i outputs $y_i^j = 1$ or $\text{det}_i^j = D$, then some honest party previously computed $\text{Verify}_{\text{VRF}}(\text{role}||\rho, \cdot, \cdot, j) = 1$.*

Proof. If an honest party P_i outputs $y_i^j = 1$ or $det_i^j = D$, then it has updated r_i^j to a value different from $d + 6$. This only happens if receives $\lambda/2$ signature chains that are valid for P_j , and this by definition 6 means that it has evaluated $\text{Verify}_{\text{VRF}}(\text{role} || \rho, y_{\text{VRF}}, \pi, j) = 1$ for some y_{VRF} and π . $\square$

In contrast to the properties proved this far, the detection and soundness properties become significantly more complex to argue in our version. In Fitzi and Nielsen’s protocol a party can simple detect all parties who signed the chain in any round earlier than it received it. But in our case, we only accept chains when they are received by a majority of the committee, and thus, some honest parties may have signed and forwarded the chain earlier. If we simply “detect” such parties, we would be breaking soundness. In order to resolve this this, it is useful to note that all honest committee members are going to send their chains in two adjacent rounds and the first round in which $\lambda/2$ chains are received by any honest party has to be one of these two rounds. So when parties update their faulty list based on the first round in which they receive a chain from the majority of the committee, then they can trust that all parties who sent a signed the chains two rounds prior must be corrupted. This adds a layer of indirection and as such we have to spend an additional round compared with Fitzi and Nielsen to maintain both soundness and detection. Thus, our constant round overhead increases by 1 compared to Elsheimy et al. who already have an extra round compared to Fitzi and Nielsen due to collecting proofs of participation in the first round.

Lemma 10 ((d)-Detection). *If for some honest party P_i , $det_i^k = D$, then an additional d parties are added to the faulty lists of all honest parties. I.e.*

$$\left| \left(\bigcap_{P_j \in \mathcal{H}} \mathcal{F}_j^* \right) \setminus \left(\bigcap_{P_j \in \mathcal{H}} \mathcal{F}_j \right) \right| \geq d.$$

Proof. If party P_i outputs $det_i = D$ then $d + 4 \leq r_i^k \leq d + 5$. P_i sets $r_i^k = r$ once it receives at least $\lambda/2$ valid signature chains by round r . By lemma 1 at least one of $\lambda/2$ chains must be sent by an honest party. Let r' be the first round in which an honest committee member sends a chain, and P_j be one such party. We will prove that the first d parties to sign the chain sent by P_j will be added to the faulty list of all honest parties and were not previously in the faulty list of all honest parties. Which in particular implies the lemma.

First note that the chain of P_j will reach all honest parties in round r' . This will prompt all other honest committee members who did not send a chain in round r' to send a chain in round $r' + 1$, which is thus the latest round that any honest party P_k can set update r_k . With this: $r_i^k \leq d + 5$ implies $r' \leq d + 5$ and that all honest parties receive a chain from all honest committee members by round $r' + 1 \leq d + 6$. This is (cf. lemma 1) at least $\lambda/2$ chains, and thus parties who have not already updated their faulty list will receive the chain of P_j and use it to update their faulty list. On the other hand: since r' was the first round in which an honest party sent a chain, no party who updates their faulty list

does so without considering the chain sent by P_j . Since $r_i^k \geq d + 4$ and we just argued that all honest parties receive $\lambda/2$ chains in round $r' + 1$: $r' \geq d + 3$. This means that $d + 3$ is the earliest round that an honest party can update their faulty list, and when they do so they have received a signature chain from P_j of length at least $d + 3$ parties and thus add at least the first d parties to sign the chain to their faulty list.

We conclude by noting that since honest parties only send valid chains and valid chains include valid PoPs for every signature: lemma 3 implies that none of the parties in the chain of P_j could have been in the faulty list of all honest parties at the start of the protocol, so the intersection of honest faulty lists grows by at least d .

□

Lemma 11 ($\mathcal{F}$ -Soundness). *If an honest party P_i outputs $\mathcal{F}_i$, then $\mathcal{F}_i$ consists only of corrupted parties*

Proof. Suppose an honest party receives a valid signature chain $\langle 1 \rangle_\sigma$ for the first time in round r and eligible to forward it. It appends its signature and forwards the chain to all parties in round $r + 1$. Consequently, all honest parties receive the chain by round $r + 1$, and by lemma 1 $\lambda/2$ of them are eligible to forward it and do so by round $r + 2$ at the latest. Thus, every honest party (in or out of the committee) receives at least $\lambda/2$ signature chains by round $r + 2$. This implies that if the first honest signer on a chain is the r -th signer, then all honest parties become the $(r + 1)$ -th signer at the latest. No honest party appears as the $(r + 2)$ -th signer or later. Now consider faulty parties' detection. Once an honest party receives $\lambda/2$ valid chains of length r in round r , it adds the first $r - 2$ signers on each chain to its faulty list. As discussed earlier, any such chain must have at most two honest signers at the end: it is either fully malicious, or all but the last one or two signers are honest. Hence, honest parties only add malicious parties to their faulty lists. □

Communication Complexity. The combined communication complexity of $\Pi_{\text{d-CoD}}^{\text{role}, \rho}$ is $O(n \cdot \lambda^3 \cdot d \cdot \kappa)$ since there are $O(\lambda)$ senders who may speak and cause separate $O(\lambda)$ -sized subcommittees to each send a message with $O(d)$ PoPs that each have size $\lambda \cdot \kappa$.

5 (0, 1)-Graded d -Detecting Agreement ($\Pi_{1\text{-GDA}}$)

In summary, $\Pi_{1\text{-GDA}}$ (see Fig. 3) is a variant of Graded Consensus protocols [14]. However, in $\Pi_{1\text{-GDA}}$, honest parties also output a list of detected malicious parties. $\Pi_{1\text{-GDA}}$ ensures that either every honest party outputs the same value y_i , or every honest party identifies at least d malicious parties (achieving d -detection).

In $\Pi_{1\text{-GDA}}$, each party has input $v_i \in \{0, 1\}$ and faulty list $\mathcal{F}_i$. Each party outputs a value $y_i \in \{0, 1\}$, a grade $g_i \in \{0, 1\}$, and an updated list of identified malicious parties $\mathcal{F}_i^* \subset \mathcal{P}$. Each party P_i invokes $\Pi_{\text{d-CoD}}^{\text{propose}, \rho}$ with input $(v_i, \mathcal{F}_i)$ to obtain output $y_i^1, \dots, y_i^n, \det_i^1, \dots, \det_i^n, \mathcal{F}_i^*$ which includes a bit “sent” by each

other party and a corresponding flag which if set to C guarantees agreement on the bit and if set to D guarantees that the honest parties commonly detect d new parties. Notably, this primitive – in order to save communication complexity – does not guarantee that the bit being input by honest parties are in general delivered to other honest parties. Only honest *committee members* will have their input reflected in the output. Conversely, if a party is not selected to speak for the role associated with $\Pi_{d\text{-CoD}}^{\text{propose},\rho}$ (identified by the string `propose|| $\rho$`) its output is guaranteed to be 0 with the correctness flag C .

This allows constructing a 1-biased graded agreement with detection by counting how many ones are received and using the correctness flags to determine the grade. By using $\lambda/2$ ones received as the threshold for outputting 1 and applying the bound from lemma 1, it is clear that validity of the output bit is guaranteed. Namely, there will be at least $\lambda/2$ honest parties on the committee, so if all honest input 1, then all honest parties receive $\lambda/2$ ones from the honest committee members. On the other hand, if there cannot be $\lambda/2$ corrupted parties on the committee, so if all honest parties input 0, no honest party receives $\lambda/2$ ones. But validity additionally requires the output *grade* to be 1 when honest parties are in pre-agreement. This is achieved by setting the grade to 1 if either $\lambda/2$ outputs are 1 with flag C or strictly more than $n - \lambda/2$ outputs are 0 with flag C .

The consistency property requires that two honest parties with grade 1 has the same output bit. This holds because the thresholds for the number of ones with flag C ($\lambda/2$) and number of zeroes with flag C ($> n - \lambda/2$) required to output grade 1 for each bit would imply that a message sent through $\Pi_{d\text{-CoD}}$ was received as two different bits at two honest parties but with flag C , which would violate the consistency property of definition 2 shown to be satisfied by $\Pi_{d\text{-CoD}}$.

Finally, to see that that protocol has d -detection, simply observe that if the output grade is different from 1, then at least one output flag of $\Pi_{d\text{-CoD}}$ was D and we can rely on its d -detection property.

Lemma 12. $\Pi_{1\text{-GDA}}$ achieves graded validity as per Definition 3.

Proof. We can assume honest faulty lists only contain corrupted parties and use validity 1 of $\Pi_{d\text{-CoD}}^{\text{propose},\rho}$ to say that the output flag of each honest party is C and corresponding output bit is identical to their input if they are elected to speak for the role `propose|| $\rho$` . By lemma 1 there are at least $\lambda/2$ honest parties on the committee. So, if all honest parties input 1, then $n_{1,C} \geq \lambda/2$ and each honest P_i would output $y_i = 1$ and $g_i = 1$. On the other hand, for each party who is not elected for role `propose|| $\rho$` , the output of $\Pi_{d\text{-CoD}}^{\text{propose},\rho}$ is 0 with flag C by validity 2 of $\Pi_{d\text{-CoD}}^{\text{propose},\rho}$ and by lemma 1 the number of corrupted parties on the committee is strictly less than $\lambda/2$. So, if all honest parties input 0, then $n_{0,C} > n - \lambda/2$ and each honest P_i would output $y_i = 0$ and $g_i = 1$. $\square$

Lemma 13. $\Pi_{1\text{-GDA}}$ achieves graded consistency as per Definition 3.

Proof. If an honest party P_i has output 1 with grade 1, then it received $\lambda/2$ ones with flag C from $\Pi_{d\text{-CoD}}^{\text{propose},\rho}$. For an honest party P_j to output 0 with any grade

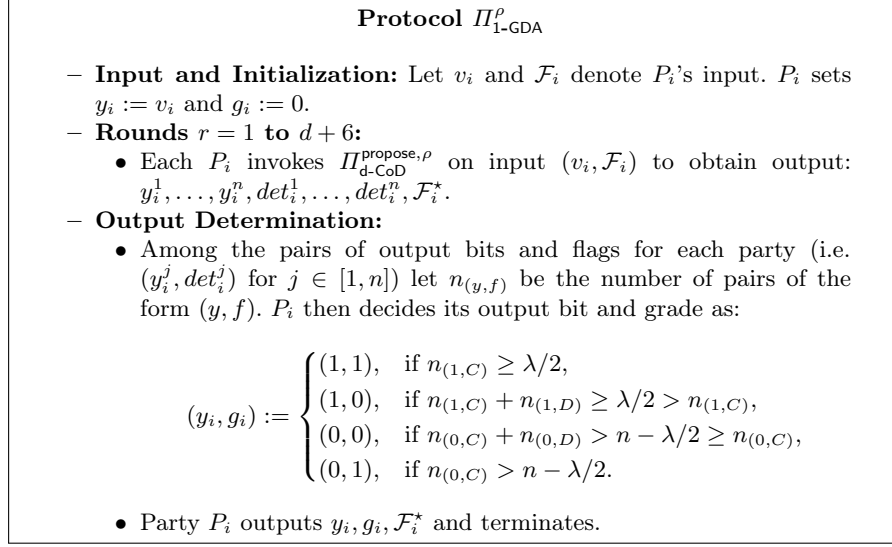


Fig. 3. Code of $\Pi_{1\text{-GDA}}^p$ for party P_i .

it would need to see strictly more than $n - \lambda/2$ zeroes from $\Pi_{d\text{-CoD}}^{\text{propose}, \rho}$. As at least one of these would be the bit sent by the same sender, this contradicts the consistency property of definition 2. Conversely, an honest party P_i with output 0 and grade 1 has received some $k > n - \lambda/2$ zeroes with flag C, no party can receive more than $n - k < \lambda/2$ ones. $\square$

Lemma 14. $\Pi_{1\text{-GDA}}$ achieves d -detection as per Definition 3.

Proof. As we have just show consistency: if two parties honest parties get different outputs, then at least one of them received grade 0. By construction of the output determination rule: grade 0 is only chosen if at least one of the flags output by $\Pi_{d\text{-CoD}}^{\text{propose}, \rho}$ is D, and we appeal to d -detection of definition 2. $\square$

Finally, we prove the main lemma of this subsection below.

Lemma 15. $\Pi_{1\text{-GDA}}$, (Fig. 3), achieves $(0, 1)$ -Graded d -Detecting Agreement as per Definition 4. Furthermore, $\Pi_{1\text{-GDA}}$ terminates in $d + 6$ rounds.

Proof. Graded validity, graded consistency, and d -detection follow from Lemmata 12, 13 and 14. We are left to prove soundness, termination and round complexity. Assume that for each honest party P_i , $P_i \notin \mathcal{F}_j$ for any honest party P_j .

Soundness. An honest party P_i inputs its initial faulty list $\mathcal{F}_i$ to $\Pi_{d\text{-CoD}}$ and according to the $\mathcal{F}$ -soundness of $\Pi_{d\text{-CoD}}$, the resulting $\mathcal{F}_i^*$ will only include malicious parties.

Termination. $\Pi_{1\text{-GDA}}$ simply runs an instance of $\Pi_{d\text{-CoD}}$ and interprets the messages to determine an output. Based on the assumption that $\Pi_{d\text{-CoD}}$ terminates,

$\Pi_{1\text{-GDA}}$ will also terminate.

Round Complexity. $\Pi_{1\text{-GDA}}$ protocol runs for $d + 6$ rounds as $\Pi_{d\text{-CoD}}$ runs for $d + 6$ rounds. $\square$

Communication Complexity. The $\Pi_{1\text{-GDA}}$ protocol is simply an instance of $\Pi_{d\text{-CoD}}$ with some extra local logic, so the communication complexity remains $O(n \cdot \lambda^3 \cdot d \cdot \kappa)$.

6 (0, 1, 2)-Graded d -Detecting Agreement ($\Pi_{2\text{-GDA}}$)

Similar to $\Pi_{1\text{-GDA}}$ protocol, $\Pi_{2\text{-GDA}}$ is a variant of Graded Consensus protocols [14], which allows honest parties to also output a list of detected malicious parties. In $\Pi_{2\text{-GDA}}$, each party starts with $v_i \in \{0, 1\}$ and faulty list $\mathcal{F}_i$. Each party outputs a value $y_i \in \{0, 1, \perp\}$, a grade $g_i \in \{0, 1, 2\}$, and an updated list of identified malicious parties $\mathcal{F}_i^* \subset \mathcal{P}$. The protocol $\Pi_{2\text{-GDA}}$ is built from $\Pi_{1\text{-GDA}}$ and the (0, 1, 2)-player-replaceable graded agreement protocol $\Pi_{2\text{-GA}}$ from [27]. Each step is executed by 2 new randomly selected committees (one for each bit), whose members remain secret until they send their first and only message.

Consequently, in the first round, each party P_i invokes $\Pi_{1\text{-GDA}}$ with input $(v_i, \mathcal{F}_i)$, storing the resulting output $(y_i^*, g_i^*, \mathcal{F}_i^*)$. To enhance the confidence on its output value, the parties run four more boosting rounds that follow the $\Pi_{2\text{-GA}}$ in [27] with y_i^* as its input. In the first boosting round, each party sends $\langle y_i^*, \rho, r \rangle_i$ to all parties if they are elected to speak for their output bit. In the second round, a new committee is elected. Each party P_i in the committee for v checks whether it received at least $\lambda/2$ messages of the form $\langle v, \rho, r - 1 \rangle_j$ for some $v \in \{0, 1\}$ in the previous round. If so, it broadcasts $\langle v, \rho, r \rangle_i$. In the third round, a new committee again checks whether it received at least $\lambda/2$ messages of the form $\langle v, \rho, r - 1 \rangle_j$ from the prior committee for some $v \in \{0, 1\}$, and did not receive any $\langle 1 - v, \rho, r - 1 \rangle_j$. If the condition holds, it sends $\langle v, \rho, r \rangle_i$ to all parties. In the fourth and final boosting round, each committee member for v verifies whether it received at least $\lambda/2$ messages of the form $\langle v, \rho, r \rangle_j$ for some $v \in \{0, 1\}$ and did not receive any $\langle 1 - v, \rho, r - 1 \rangle_j$. If so, it broadcasts $\langle v, \rho, r \rangle_i$. At the end of the boosting rounds, each party P_i determines its final output value and grade based on the number of received messages of the form $\langle v, \rho, r \rangle_j$.

Since the sets S'_v and S''_v are composed of older voting messages, we must ensure that every set has been correctly formed. In particular, we need to verify that each message contained in any of these sets was sent by a party that was part of the corresponding elected committee. For a modular verification of these conditions, we formally define a validity predicate below.

Definition 7 (Boosting Validity Predicate V). We call a PPT predicate $V: \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}$ a boosting validity predicate for value v . Let $r, \rho \geq 1$, $v \in \{0, 1\}$ and let M be a set of messages, where each message is of the form $(\langle m, \rho, r \rangle_j, y_{\text{VRF}_j}, \pi_j)$. We define $V(M, (\text{boost}_s || \rho || v)) = 1$ if and only if

$$- |M| \geq \lambda/2$$

- For every $(\langle m, \rho, r \rangle_j, y_{\text{VRF}_j}, \pi_j) \in M$, $\text{Verify}_{\text{VRF}}((\text{boost}_s \parallel \rho \parallel v), y_{\text{VRF}_j}, \pi_j, j) = 1$

Note that the output $\mathcal{F}_i^*$ is the faulty list output from $\Pi_{1\text{-GDA}}$ and does not get updated further. The protocol runs for $d + 10$ rounds: $d + 6$ for $\Pi_{1\text{-GDA}}$ and 4 additional rounds for the boosting rounds.

Lemma 16. *Let an P_i be an honest party that receives $(\langle v, \rho, r \rangle_i, y_{\text{VRF}_j}, \pi_j)$ by the end of round $d + 10$ such that $\text{Verify}_{\text{VRF}}((\text{boost}_4 \parallel \rho \parallel v), y_{\text{VRF}_j}, \pi_j, j) = 1$ along with set S'_v such that $V(S'_v, (\text{boost}_3 \parallel \rho \parallel v)) = 1$. Then, except with negligible probability, no honest party P_k receives $(\langle 1 - v, \rho, r \rangle_i, y_{\text{VRF}_j}, \pi_j)$ accompanied with set S'_{1-v} satisfying the same conditions.*

Proof. Let P_i be an honest party that receives a message of the form $(\langle v, \rho, r \rangle_i, y_{\text{VRF}_j}, \pi_j)$ by the end of round $d + 10$, such that $\text{Verify}_{\text{VRF}}((\text{boost}_4 \parallel \rho \parallel v), y_{\text{VRF}_j}, \pi_j, j) = 1$ along with set S'_v such that $V(S'_v, (\text{boost}_3 \parallel \rho \parallel v)) = 1$. According to Definition 7, the predicate $V(S'_v, (\text{boost}_3 \parallel \rho \parallel v))$ holds only if S'_v consists of at least $\lambda/2$ messages of the form $(\langle v, \rho, r \rangle_j, y_{\text{VRF}_j}, \pi_j)$ that were sent by the elected committee during round $d + 9$. By Lemma 1, at least one of these messages must originate from an honest party. According to the protocol, an honest committee member sends a message $(\langle v, \rho, r \rangle_i, y_{\text{VRF}_j}, \pi_j)$ along with S'_v in round $d + 9$ only if it had received at least $\lambda/2$ messages $(\langle v, \rho, r \rangle_j, y_{\text{VRF}_j}, \pi_j)$ accompanied by set S_v from the committee in round $d + 8$, and it did not receive any message of the form $(\langle 1 - v, \rho, r \rangle_j, y_{\text{VRF}_j}, \pi_j)$ accompanied with set S_{1-v} . This implies that no honest party sends $\langle 1 - v, \rho, r \rangle_j$ in round $d + 8$. As a result, no party can construct a valid set S'_{1-v} satisfying $V(S'_{1-v}, (\text{boost}_2 \parallel \rho \parallel v)) = 1$, or a valid S''_{1-v} satisfying $V(S''_{1-v}, (\text{boost}_3 \parallel \rho \parallel v)) = 1$, since fewer than $\lambda/2$ committee members can be malicious (except with negligible probability from Lemma 1). Therefore, no honest party receives a message $(\langle 1 - v, \rho, r \rangle_j, y_{\text{VRF}_j}, \pi_j)$ such that $\text{Verify}_{\text{VRF}}((\text{boost}_4 \parallel \rho \parallel 1 - v), y_{\text{VRF}_j}, \pi_j, j) = 1$ along with set S''_{1-v} such that $V(S''_{1-v}, (\text{boost}_3 \parallel \rho \parallel v)) = 1$.

Lemma 17. *Assume $\Pi_{1\text{-GDA}}^\rho$, (Fig. 3) achieves $(0, 1)$ -Graded d -Detecting Agreement as per Definition 4. $\Pi_{2\text{-GDA}}^\rho$ achieves graded validity per Definition 4.*

Proof. Assume that for all honest parties P_i , the initial input is $v_i = v$. Moreover, assume that no honest party appears in the faulty list of any other honest party, i.e., $P_i \notin \mathcal{F}_j$ for all honest P_i and P_j . Each honest party P_i invokes the protocol $\Pi_{1\text{-GDA}}^\rho$ with input $(v, \mathcal{F}_i)$. By the graded validity property of $\Pi_{1\text{-GDA}}$ (Definition 3), all honest parties output $(v, 1, \mathcal{F}_i^*)$. In the subsequent first boosting round, a new committee is elected. By Lemma 1, at least $\lambda/2$ of the elected members are honest, and fewer than $\lambda/2$ are malicious. Each honest elected committee member sends the message $\langle v, \rho, r \rangle$ to all parties. Consequently, every honest party receives at least $\lambda/2$ such messages and forms the set S_v . Since there are fewer than $\lambda/2$ malicious members and all honest members send $\langle v, \rho, r \rangle$, no honest party receives $\lambda/2$ messages of the form $\langle 1 - v, \rho, r \rangle$, and thus no set S_{1-v} of size $\geq \lambda/2$ is formed. In the second boosting round, a new committee

is elected. Again, by Lemma 1, at least $\lambda/2$ members are honest. These honest members send $\langle v, \rho, r \rangle$ along with set S_v to all parties. Each honest party collects at least $\lambda/2$ such messages and forms the set S'_v . As before, no set S'_{1-v} of size $\geq \lambda/2$ is formed due to the number of malicious committee members. Similarly, in the third boosting round, another committee is elected. The honest committee members send $\langle v, \rho, r \rangle$ along with set S'_v to all parties. By Lemma 1, each honest party receives at least $\lambda/2$ messages of the form $\langle v, \rho, r \rangle$, and forms the set S''_v . Finally, in the fourth boosting round, another committee is elected. This honest committee members send $\langle v, \rho, r \rangle$ along with set S''_v to all parties. By Lemma 1, each honest party receives at least $\lambda/2$ messages of the form $\langle v, \rho, r \rangle$ along with S''_v . As a result, all honest parties decide on $y_i = v$ and assign grade $g_i = 2$.

Lemma 18. Assume $\Pi_{1\text{-GDA}}^\rho$, (Fig. 3) achieves $(0, 1)$ -Graded d -Detecting Agreement as per Definition 4. $\Pi_{2\text{-GDA}}^\rho$ achieves graded consistency per Definition 4.

Proof. Assume that for all honest parties P_i , $v_i = v$. Further, assume that for each honest party P_i , $P_i \notin \mathcal{F}_j$ for any honest party P_j . We need to show that (1) $|g_i - g_j| \leq 1$ and (2) $g_i, g_j \geq 1$ implies that $y_i = y_j$. For the first, we need prove if an honest party output $g_i = 2$, then no honest party outputs grade $g_i = 0$. Let P_i be an honest party that outputs $g_i = 2$, we will prove that every other honest party P_j outputs $g_j \geq 1$. An honest party P_i outputs grade $g_i = 2$ and $y_i = v$ if it receives at least $\lambda/2$ messages of the form $\langle v, \rho, r \rangle$ along with set S''_v . As the number of malicious parties on a committee is at less than $\lambda/2$ by Lemma 1, at least one of these messages came from an honest party. Thus, every honest party receives at least a message of the form $\langle v, \rho, r \rangle$, and does not receive a message $(\langle 1 - v, \rho, r \rangle_j, y_{\text{VRF}_j}, \pi_j)$ such that $\text{Verify}_{\text{VRF}}((\text{boost}_4 || \rho || 1 - v), y_{\text{VRF}_j}, \pi_j, j) = 1$ along with set S''_{1-v} such that $V(S''_{1-v}, (\text{boost}_3 || \rho || v)) = 1$ by Lemma 16. Thus, every honest party outputs $g_i \geq 1$ and $y_i = v$. Next, we need to prove that if $g_i, g_j \geq 1$, $v_i = v_j$. According to the protocol, an honest party P_i outputs $g_i = 1$ and $y_i = v$, if it receives a message of the form $\langle v, \rho, r \rangle_i$ along with set S''_v by the end of round $d + 10$. similarly, by Lemma 16, no honest party receives a message $(\langle 1 - v, \rho, r \rangle_j, y_{\text{VRF}_j}, \pi_j)$ such that $\text{Verify}_{\text{VRF}}((\text{boost}_4 || \rho || 1 - v), y_{\text{VRF}_j}, \pi_j, j) = 1$ along with set S''_{1-v} such that $V(S''_{1-v}, (\text{boost}_3 || \rho || v)) = 1$.

Lemma 19. Assume $\Pi_{1\text{-GDA}}^\rho$, (Fig. 3) achieves $(0, 1)$ -Graded d -Detecting Agreement as per Definition 4. $\Pi_{2\text{-GDA}}^\rho$ achieves $(0, 1, 2)$ -Graded Faulty-Detecting Byzantine Agreement per Definition 4.

Proof. Assume that for each honest party P_i , $P_i \notin \mathcal{F}_j$ for any honest party P_j . Suppose that every honest party P_i inputs $(v_i, \mathcal{F}_i)$ to $\Pi_{2\text{-GDA}}$, where $v_i \in \{0, 1\}$ and $\mathcal{F}_i \subset \mathcal{P}$.

Graded consistency and graded validity follow from Lemmata 18 and 17.

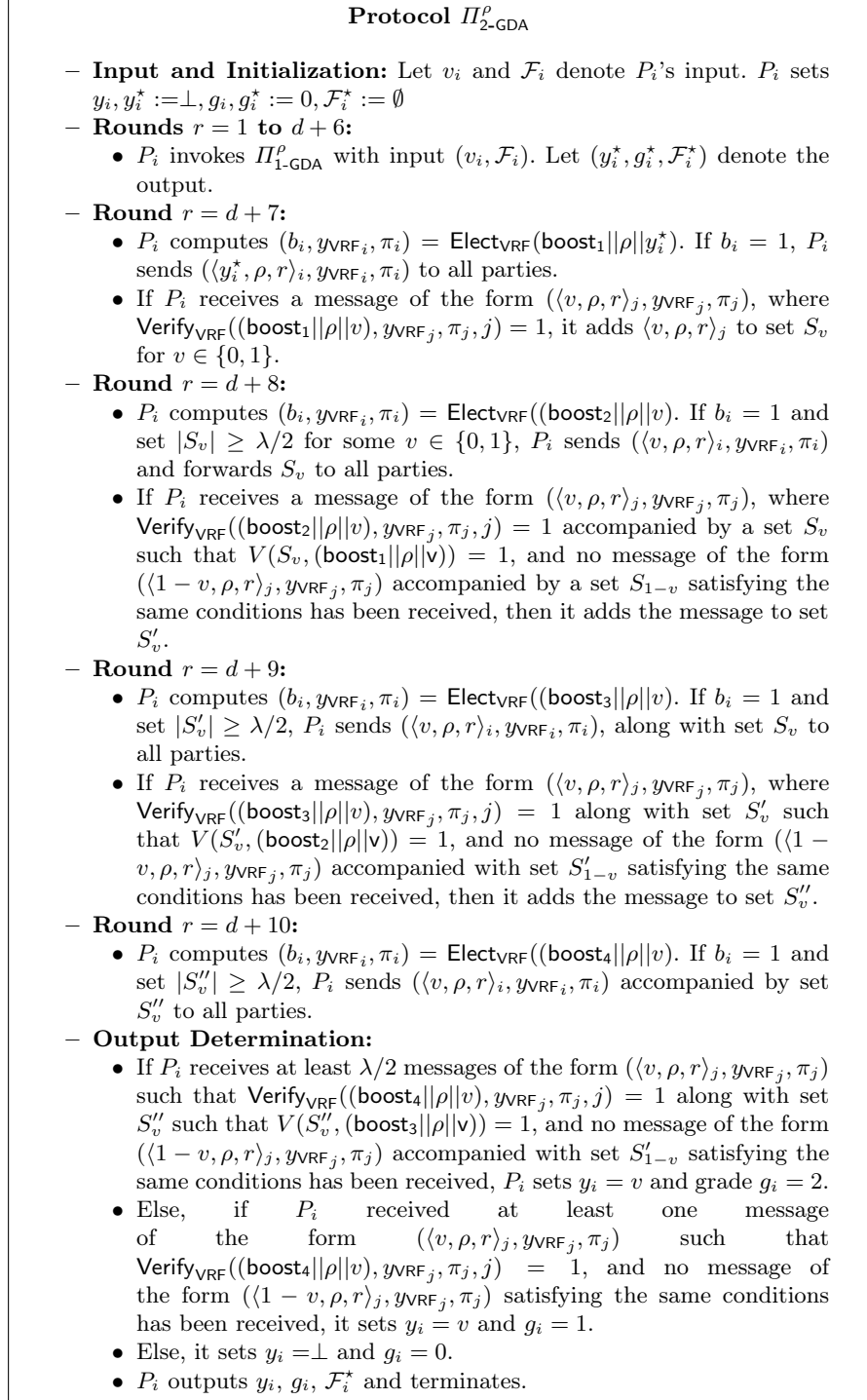
d -Detection. Assume an honest party p_i outputs a $g_i < 2$. If an honest party p_i outputs a $g_i < 2$, it follows from graded validity of $\Pi_{2\text{-GA}}$ that not all parties input the same value to $\Pi_{2\text{-GA}}$. Parties invoke $\Pi_{2\text{-GA}}$ with the output value they obtained from $\Pi_{1\text{-GDA}}$, so there must be two honest parties P_i and P_j that output distinct values y_i^* and y_j^* from $\Pi_{1\text{-GDA}}$. Thus, d -detection of $\Pi_{2\text{-GDA}}$ is directly

implied by d -detection of $\Pi_{1\text{-GDA}}$.

Soundness. The output faulty list, denoted as $\mathcal{F}_i^*$, is based on the output faulty list from $\Pi_{1\text{-GDA}}$. Due to the soundness property of $\Pi_{1\text{-GDA}}$, $\mathcal{F}_i^*$ contains only malicious parties.

Termination: The protocol terminates in $d+10$ rounds; $d+6$ rounds of $\Pi_{1\text{-GDA}}$, and followed by 4 synchronous boosting rounds.

Communication Complexity. The communication complexity of $\Pi_{2\text{-GDA}}$ is that of the $\Pi_{\text{d-CoD}}$ ($O(n \cdot \lambda^3 \cdot d \cdot \kappa)$) in addition to the 4 rounds of boosting. In each round, $O(\lambda)$ parties send $O(\lambda)$ signatures of size κ to all n parties, resulting in $O(n \cdot \lambda^2 \cdot \kappa)$ bits of communication. The dominating term is $O(n \cdot \lambda^3 \cdot d \cdot \kappa)$.

**Fig. 4.** Code of $\Pi_{2\text{-GDA}}$ for party P_i .

7 Byzantine Agreement Protocol

In this subsection, we present our randomized Byzantine agreement protocol which has expected constant time and $O(n \cdot \text{poly}(\lambda, \kappa))$ communication complexity and, with negligible error, always terminates in $(1 + \epsilon)f$ rounds and incurs $O(\min(f \cdot n \cdot \text{poly}(\lambda, \kappa), t \cdot n \cdot \text{poly}(\lambda, \kappa)))$ communication complexity for corruption $f \leq t$. We demonstrate how to construct the randomized early-stopping Byzantine agreement protocol, Π_{BA} . In Π_{BA} , each party starts with an input value $v_i \in \{0, 1\}$ and outputs an output value $y_i \in \{0, 1\}$. Π_{BA} runs in iterations. In each iteration k , parties run $\Pi_{2\text{-GDA}}$ with input $(v_i, \mathcal{F}_i)$. Consequently, each party P_i stores the output $(y_{\text{GDA}_i}, g_i, \mathcal{F}_i)$ of $\Pi_{2\text{-GDA}}$. Based on the grade g_i obtained from $\Pi_{2\text{-GDA}}$, each party P_i determines whether it is safe to terminate. If P_i outputs $g_i < 2$, it indicates that it is not safe to terminate, and more iterations are run to reach agreement.

Conversely, if P_i outputs $g_i = 2$, it is confident that all other honest parties P_j output the same value $y_{\text{GDA}_i} = y_{\text{GDA}_j}$ due to the graded consistency of $\Pi_{2\text{-GDA}}$. Each Party P_i updates its input value for the next iteration based on the output grade $g_i \in \{0, 1, 2\}$ of $\Pi_{2\text{-GDA}}$. If $g_i > 0$, it updates its input value to the next iteration based on the output value $y_{\text{GDA}_i} \in \{0, 1\}$ of $\Pi_{2\text{-GDA}}$, setting $v_i = y_{\text{GDA}_i}$. Otherwise, if $g_i = 0$, it sets its input value to the next iteration based on the random coin output from Π_{CC}^P protocol.

In Π_{CC}^P , each party is independently elected with probability λ/n . If a party P_i is elected—that is, if $\text{Elect}_{\text{VRF}}(\text{randomize} \parallel \rho) = (1, y_{\text{VRF}}, \pi)$ it sends the message $\langle \rho, y_{\text{VRF}}, \pi \rangle_i$ to all parties. Upon receiving messages of the form $\langle \rho, y_{\text{VRF}_j}, \pi_j \rangle_j$, party P_i verifies each using $\text{Verify}_{\text{VRF}}(\text{randomize} \parallel \rho, j)$. It then defines c_i by taking the minimal VRF output among the valid messages: y_{VRF}^* and outputs its least significant bit. Since there are more honest than corrupted parties on the committee (cf. lemma 1) and the VRF outputs are indistinguishable from random uniform strings, the probability that the least one is honest is at the output of an honest party is at least $1/2$. Whenever this happens: every honest party receives the same common output and this output is additionally a uniform random random bit.

Lemma 20. *If $t < (1 - \alpha)n/2$, then with probability at least $1/2$: the output of Π_{CC}^P is the same for all honest parties and computationally indistinguishable from a uniform random bit before the protocol begins.*

Proof. By lemma 1 and assumption $t < (1 - \alpha)n/2$, there are at least $\lambda/2$ parties who will send a valid string $\langle \rho, y_{\text{VRF}}, \pi \rangle_i$ to all parties while they are honest, and strictly less than $\lambda/2$ parties who are corrupted before sending a valid string. So, among the parties who may send a valid message of the form $\langle \rho, y_{\text{VRF}}, \pi \rangle_i$, the majority are honest. Since each VRF output y_{VRF} is indistinguishable from a uniform random string, the probability that the lowest value y_{VRF}^* is sent by a party who is honest at the time of sending is at least $1/2$. Whenever this happens, y_{VRF}^* is received by all honest parties, and they all use its least significant bit as output. At the same time, the sender being honest at the time of sending

y_{VRF}^* means that the value is unpredictable to the adversary. (Note that the adversary cannot take the message back post-corruption or produce an alternative VRF output for the role.) Finally, while y_{VRF}^* is by the definition of fig. 7 indistinguishable from a uniform random string of length ℓ_{VRF} , it is a priori not sampled independently from other outputs. It is selected as the minimum among the VRF outputs of all committee members, some of which may be corrupted. But note that the least significant bit of y_{VRF}^* is independently sampled *unless* it influences this selection. That would only be the case if some other output y_{VRF}' is a collision on all other bits, which happens with probability less than $n \cdot 2^{\ell_{\text{VRF}}-1}$, i.e. negligible probability in ℓ_{VRF} which dominates the statistical security parameter λ). $\square$

Therefore, by integrating $\Pi_{2\text{-GDA}}$ and Π_{CC} , each iteration consists of $d + 10$ rounds. Therefore, the overall round complexity of Π_{BA} depends on the number of iterations it runs. We demonstrate in Lemma 25 that it has expected constant number of rounds and, except with negligible probability, always terminates in a number of iterations that is a function of f .

Protocol Π_{CC}^ρ
– Input and Initialization: P_i sets $c_i := 0$ • Round 1: * Each party P_i computes $(b, y_{\text{VRF}}, \pi) = \text{Elect}_{\text{VRF}}(\text{randomize} \rho)$. If $b = 1$ it sends $\langle y_{\text{VRF}}, \pi \rangle_i$ to all parties. – Output Determination: Among all received messages of the form $\langle y_{\text{VRF}_j}, \pi_j \rangle_j$ which are signed correctly under pk_j and where $\text{Verify}_{\text{VRF}}(\text{randomize} \rho, y_{\text{VRF}_j}, \pi_j, j) = 1$, each P_i picks the minimum VRF output y_{VRF}^* and outputs its least significant bit $c_i = \text{Lsb}(y_{\text{VRF}}^*)$.

Fig. 5. Code of Π_{BA}^r for party P_i

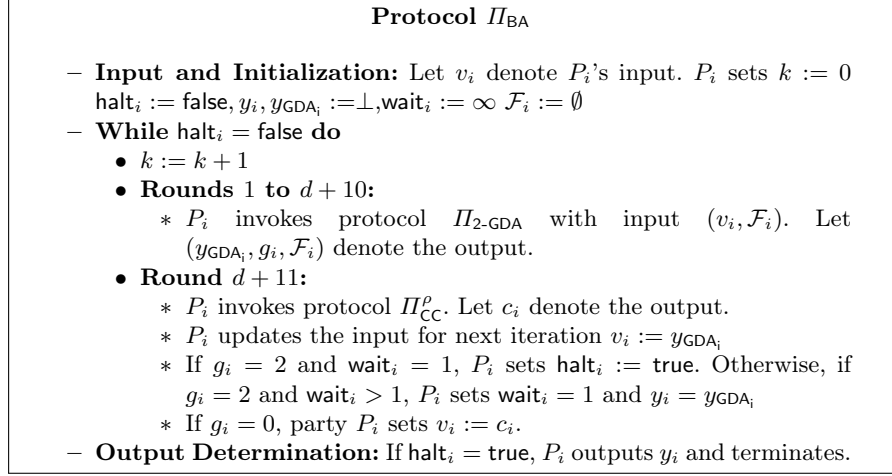
Similar to $\Pi_{1\text{-GDA}}$, we also establish that honest parties are never included in the faulty lists of other honest parties in any iteration, which is a needed assumption for $\Pi_{2\text{-GDA}}$

Lemma 21. *At the start of each iteration, the faulty list $\mathcal{F}_i$ of every honest party P_i contains only corrupted parties.*

Proof. The proof follows from the soundness property of $\Pi_{2\text{-GDA}}$ (Lemma 19). $\square$

We proceed to prove both validity and agreement of Π_{BA} .

Lemma 22. *Except for negligible probability, Π_{BA} achieves validity per Definition 1*

Fig. 6. Code of Π_{BA^r} for party P_i

Proof. Assume all honest parties have the same initial value ($v_i = v$). Every party invokes $\Pi_{2\text{-GDA}}$ in the second round with input $(v_i, \mathcal{F}_i)$. From graded validity of $\Pi_{2\text{-GDA}}$, every honest party outputs $(y_{\text{GDA}_i} = v, g_i = 2, \mathcal{F}_i)$. Consequently, as $g_i = 2$, every honest party sets $\text{wait}_i := 1$ and $y_i = v$. In the next iteration, each honest party outputs $y_i = v$ and terminates. $\square$

Lemma 23. *If all honest parties set y_{GDA_i} to the same value in iteration k , then all honest parties will terminate by at most iteration $k + 2$.*

Proof. Let all honest parties P_i set y_{GDA_i} to the same value v in iteration k , i.e., $y_{\text{GDA}_i} := v$. If a party has grade $g_i = 2$, it sets $\text{wait}_i := 1$ and $y_i := v$. Otherwise, it does nothing. Consequently, each party updates its input value v_i for the subsequent iteration based on this output value of $\Pi_{1\text{-GDA}}$, such that $v_i := v$. In the next iteration ($k + 1$), all honest parties invoke $\Pi_{2\text{-GDA}}$ with the same input value v . According to the graded validity of $\Pi_{2\text{-GDA}}$, all honest parties set $g_i := 2$. If $\text{wait}_i = 1$, P_i sets $\text{halt}_i = \text{true}$, outputs $y_i = v$ and terminates in iteration $k + 1$. Otherwise, each other honest party sets $\text{wait}_i = 1$ and $y_i := v$. In iteration $k + 2$, as $\text{wait}_i = 1$, each honest party sets $\text{halt}_i := \text{true}$, outputs $y_i = v$ and terminates.

Lemma 24. *Except for negligible probability,, Π_{BA} achieves agreement per Definition 1*

Proof. Let P_i be the first honest party to set $\text{wait}_i = 1$ and $y_i := v$ in the earliest iteration, say $k > 0$, indicating it will wait for one more iteration before terminating. This happens when P_i sets wait_i to 1, a condition met if its g_i equals 2, determined by the output of $\Pi_{2\text{-GDA}}$. By the graded consistency of $\Pi_{2\text{-GDA}}$, every other honest party P_j outputs $y_{\text{GDA}_j} = v$ and $g_j \geq 1$. As a result, every honest party updates its input variable for the next iteration to $v_i = v$.

According to Lemma 23, every honest party terminate with output $y_i = v$ by the latest in iteration $k + 2$. $\square$

Finally, we prove the round complexity of Π_{BA} .

Lemma 25. *Π_{BA} has expected constant time and, except with negligible probability, always termination within $(d + 11) \cdot (\lfloor f/d \rfloor + 2)$ rounds.*

Proof. First, we prove expected constant time. If an honest party P_i has $g_i = 2$ by the end of iteration k , all honest parties terminate by the end of iteration $k+2$. So, let's assume every honest party has $g_i < 2$ by iteration k . Then, from Lemma 20, with probability at least $1/2 \cdot 1/2 = 1/4$, the common coin value $c_j^{(k)}$ of all honest parties $P_j \in \mathcal{P}$ is equal to the output y_{GDA_i} of honest parties P_i with $g_i = 1$. Note, if $g_i, g_j = 1$ for honest parties P_i and P_j , then $y_{\text{GDA}_i} = y_{\text{GDA}_j}$ from graded consistency of $\Pi_{2\text{-GDA}}$. Thus, all honest parties start the next iteration with the same value. From Lemma 23, all honest parties terminate by iteration $k + 2$. Thus, the exact round complexity in expectation is $(\frac{1}{1/4} + 2)(d + 11) = 6(d + 11)$.

Next, we prove the always terminating within $(d + 11) \cdot (\lfloor f/d \rfloor + 2)$ rounds, except with negligible probability. A party P_i terminates in iteration $k + 1$ after setting its g_i to 2 in iteration k , which is based on the output of $\Pi_{2\text{-GDA}}$. According to the graded consistency of $\Pi_{2\text{-GDA}}$, except for negligible probability, $y_{\text{GDA}_i} = y_{\text{GDA}_j}$ for every honest parties P_i and P_j in iteration k . Consequently, all other honest parties terminate in iteration $k + 2$ from Lemma 23. The setting of g_i by a party is based on the result of $\Pi_{2\text{-GDA}}$. If an honest party P_i sets $g_i < 2$, then at least d new parties are added to the faulty list of all honest parties P_i according to the d -detection property of $\Pi_{2\text{-GDA}}$. Thus, since there are f faulty parties, there can be at most $\lfloor f/d \rfloor$ many iterations where all honest parties output $g_i < 2$. Thus, after at most $\lfloor f/d \rfloor + 1$ many iterations, all honest parties set $g_i = 2$, followed by one additional iteration for all honest parties to terminate. Therefore, except for negligible probability, the total round complexity is $(d+11) \cdot (\lfloor f/d \rfloor + 2)$.

We summarize the preceding lemmata into the main theorem of this section:

Theorem 1. *Assume a PKI setup, random common coin, and $t < n/2$. Π_{BA} (Fig. 6) achieves Byzantine Agreement per Definition. 1. Furthermore, Π_{BA} terminates in expected constant time and has expected communication complexity $O(n \cdot \lambda^3 \cdot \kappa)$, while except with negligible probability in λ it runs for at most $(d+11) \cdot (\lfloor f/d \rfloor + 2)$ rounds with communication bounded by $O(n \cdot \min(f, \lambda) \cdot \lambda^3 \cdot \kappa)$ for any execution with $f \leq t$ corrupted parties.*

Proof. The theorem follows from the preceding lemmata. For the communication complexity, we established that the protocol $\Pi_{2\text{-GDA}}$ has communication complexity $O(n \cdot d \cdot \lambda^3 \cdot \kappa)$ where d falls out when it is a fixed a constant. This dominates the Π_{CC} in which $O(\lambda)$ parties send $O(\kappa)$ bits to n parties. As there are no further costs and the BA terminates with constant probability in each phase, its expected communication complexity is $O(n \cdot \lambda^3 \cdot \kappa)$. For the bounds depending on the actual number of corruptions $f \leq t$, we established in lemma 25 (beyond

the stated round complexity) that it runs for at most $\lceil f/d \rceil + 2$ phases. As it additionally terminates with a constant probability in every phase, its maximum number of phases is $O(\min(f, \lambda))$ except with negligible probability in λ . Combining with the communication bound for each phase we get $O(\min(f, \lambda) \cdot n \cdot \lambda^3 \cdot \kappa)$.

References

1. Abraham, I., Chan, T.H.H., Dolev, D., Nayak, K., Pass, R., Ren, L., Shi, E.: Communication complexity of byzantine agreement, revisited. pp. 317–326 (2019). <https://doi.org/10.1145/3293611.3331629>
2. Abraham, I., Chan, T.H.H., Dolev, D., Nayak, K., Pass, R., Ren, L., Shi, E.: Communication complexity of byzantine agreement, revisited (2020)
3. Abraham, I., Dolev, D.: Byzantine agreement with optimal early stopping, optimal resilience and polynomial complexity. In: Proceedings of the Forty-Seventh Annual ACM Symposium on Theory of Computing. p. 605–614. STOC '15, Association for Computing Machinery, New York, NY, USA (2015). <https://doi.org/10.1145/2746539.2746581>
4. Berman, P., Garay, J.A., Perry, K.J.: Optimal early stopping in distributed consensus. In: Segall, A., Zaks, S. (eds.) Distributed Algorithms. pp. 221–237. Springer Berlin Heidelberg, Berlin, Heidelberg (1992)
5. Chan, T.H.H., Pass, R., Shi, E.: Sublinear-round byzantine agreement under corrupt majority. In: IACR International Conference on Public-Key Cryptography. pp. 246–265. Springer (2020)
6. Chen, J., Micali, S.: Algorand (2017), <https://arxiv.org/abs/1607.01341>
7. David, B., Gazi, P., Kiayias, A., Russell, A.: Ouroboros praos: An adaptively-secure, semi-synchronous proof-of-stake blockchain. pp. 66–98 (2018). https://doi.org/10.1007/978-3-319-78375-8_3
8. Deligios, G., Klasovita, I., Liu-Zhang, C.D.: Asymptotically optimal early termination for dishonest majority broadcast. pp. 116–144 (2025). https://doi.org/10.1007/978-3-031-91092-0_5
9. Dolev, D., Reischuk, R.: Bounds on information exchange for byzantine agreement. J. ACM **32**(1), 191–204 (jan 1985). <https://doi.org/10.1145/2455.214112>, <https://doi.org/10.1145/2455.214112>
10. Dolev, D., Reischuk, R., Strong, H.R.: Early stopping in byzantine agreement. J. ACM **37**(4), 720–741 (oct 1990). <https://doi.org/10.1145/96559.96565>
11. Dolev, D., Strong, H.: Authenticated algorithms for byzantine agreement. SIAM J. Comput. **12**, 656–666 (11 1983). <https://doi.org/10.1137/0212045>
12. Elsheimy, F., Loss, J., Papamanthou, C.: Early stopping byzantine agreement in $(1 + \epsilon) \cdot f$ rounds. pp. 398–424 (2024). https://doi.org/10.1007/978-981-96-0938-3_13
13. Elsheimy, F., Loss, J., Papamanthou, C.: Towards optimal early stopping agreement protocols. Cryptology ePrint Archive, Report 2025/246 (2025), <https://eprint.iacr.org/2025/246>
14. Feldman, P., Micali, S.: Optimal algorithms for byzantine agreement. In: Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing. p. 148–161. STOC '88, Association for Computing Machinery, New York, NY, USA (1988). <https://doi.org/10.1145/62212.62225>
15. Feldman, P., Micali, S.: Optimal algorithms for byzantine agreement. pp. 148–161 (1988). <https://doi.org/10.1145/62212.62225>

16. Fitzi, M., Nielsen, J.: On the number of synchronous rounds sufficient for authenticated byzantine agreement. In: Distributed Computing. pp. 449–463. Springer Berlin Heidelberg, Berlin, Heidelberg (09 2009). https://doi.org/10.1007/978-3-642-04355-0_46
17. Garay, Moses, Y.: Fully polynomial byzantine agreement for $n > 3t$ processors in $t + 1$ rounds. SIAM Journal on Computing **27**(1), 247–290 (1998). <https://doi.org/10.1137/S0097539794265232>
18. Garay, J.A., Moses, Y.: Fully polynomial byzantine agreement in $t + 1$ rounds. In: Proceedings of the Twenty-Fifth Annual ACM Symposium on Theory of Computing. p. 31–41. STOC '93, Association for Computing Machinery, New York, NY, USA (1993). <https://doi.org/10.1145/167088.167101>
19. Ghinea, D., Goyal, V., Liu-Zhang, C.D.: Round-optimal byzantine agreement. pp. 96–119 (2022). https://doi.org/10.1007/978-3-031-06944-4_4
20. Karlin, A., Yao, A.: Probabilistic lower bounds for byzantine agreement and clock synchronization. Unpublished (1984)
21. Katz, J., Koo, C.Y.: On expected constant-round protocols for byzantine agreement. In: Dwork, C. (ed.) Advances in Cryptology - CRYPTO 2006. pp. 445–462. Springer Berlin Heidelberg, Berlin, Heidelberg (2006)
22. King, V., Saia, J.: Breaking the $o(n^2)$ bit barrier: Scalable byzantine agreement with an adaptive adversary (2010)
23. Lamport, L., Shostak, R., Pease, M.: The byzantine generals problem. ACM Trans. Program. Lang. Syst. **4**(3), 382–401 (jul 1982). <https://doi.org/10.1145/357172.357176>
24. Loss, J., Nielsen, J.B.: Early stopping for any number of corruptions. pp. 457–488 (2024). https://doi.org/10.1007/978-3-031-58734-4_16
25. Micali, S., Rabin, M., Vadhan, S.: Verifiable random functions. In: 40th Annual Symposium on Foundations of Computer Science (Cat. No.99CB37039). pp. 120–130 (1999). <https://doi.org/10.1109/SFFCS.1999.814584>
26. Micali, S.: Very simple and efficient byzantine agreement. In: Papadimitriou, C.H. (ed.) 8th Innovations in Theoretical Computer Science Conference, ITCS 2017, January 9–11, 2017, Berkeley, CA, USA. LIPIcs, vol. 67, pp. 6:1–6:1. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2017). <https://doi.org/10.4230/LIPICS.ITCS.2017.6>
27. Micali, S., Vaikuntanathan, V.: Optimal and player-replaceable consensus with an honest majority (2017)
28. Perry, K., Toueg, S.: An authenticated byzantine generals algorithm with early stopping. Cornell eCommons (1984), online
29. Wan, J., Xiao, H., Shi, E., Devadas, S.: Expected constant round byzantine broadcast under dishonest majority. pp. 381–411 (2020). https://doi.org/10.1007/978-3-030-64375-1_14
30. Zamsky, A.: A randomized byzantine agreement protocol with constant expected time and guaranteed termination in optimal (deterministic) time. pp. 201–208 (1996). <https://doi.org/10.1145/248052.248091>

A VRF

For completeness we include the description of $\mathcal{F}_{\text{VRF}}$ by David et al. [7].

Functionality $\mathcal{F}_{\text{VRF}}$

$\mathcal{F}_{\text{VRF}}$ interacts with parties $P_1, \dots, P_n$ as follows:

- **Key Generation.** Upon receiving a message $(\text{KeyGen}, \text{sid})$ from a party P_i , hand $(\text{KeyGen}, \text{sid}, P_i)$ to the adversary. Upon receiving $(\text{VerificationKey}, \text{sid}, P_i, v)$ from the adversary, if P_i is honest, verify that v is unique, record the pair (P_i, v) and return $(\text{VerificationKey}, \text{sid}, v)$ to P_i . Initialize the table $T(v, \cdot)$ to empty.
- **Malicious Key Generation.** Upon receiving a message $(\text{KeyGen}, \text{sid}, v)$ from $\mathcal{S}$, verify that v has not being recorded before; in this case initialize table $T(v, \cdot)$ to empty and record the pair $(\mathcal{S}, v)$.
- **VRF Evaluation.** Upon receiving a message $(\text{Eval}, \text{sid}, m)$ from P_i , verify that some pair (P_i, v) is recorded. If not, then ignore the request. Then, if the value $T(v, m)$ is undefined, pick a random value y from $\{0, 1\}^{\ell_{\text{VRF}}}$ and set $T(v, m) = (y, \emptyset)$. Then output $(\text{Evaluated}, \text{sid}, y)$ to P_i , where y is such that $T(v, m) = (y, S)$ for some S .
- **VRF Evaluation and Proof.** Upon receiving a message $(\text{EvalProve}, \text{sid}, m)$ from P_i , verify that some pair (P_i, v) is recorded. If not, then ignore the request. Else, send $(\text{EvalProve}, \text{sid}, P_i, m)$ to the adversary. Upon receiving $(\text{EvalProve}, \text{sid}, m, \pi)$ from the adversary, if value $T(v, m)$ is undefined, verify that π is unique, pick a random value y from $\{0, 1\}^{\ell_{\text{VRF}}}$ and set $T(v, m) = (y, \pi)$. Else, if $T(v, m) = (y, S)$, set $T(v, m) = (y, S \cup \pi)$. In any case, output $(\text{Evaluated}, \text{sid}, y, \pi)$ to P_i .
- **Malicious VRF Evaluation.** Upon receiving a message $(\text{Eval}, \text{sid}, v, m, \pi)$ from $\mathcal{S}$ for some v , do the following. First, if $(\mathcal{S}, v)$ is recorded and $T(v, m)$ is undefined, then choose a random value y from $\{0, 1\}^{\ell_{\text{VRF}}}$ and set $T(v, m) = (y, S)$ and output $(\text{Evaluated}, \text{sid}, y)$ to $\mathcal{S}$. The same is performed in case (P_i, v) is recorded and P_i corrupted. Else, if $T(v, m) = (y, S')$ for some $S' \neq \emptyset$, union S to S' and output $(\text{Evaluated}, \text{sid}, y)$ to $\mathcal{S}$, else ignore the request.
- **Verification.** Upon receiving a message $(\text{Verify}, \text{sid}, m, y, \pi, v')$ from some party P_i , send $(\text{Verify}, \text{sid}, m, y, \pi, v')$ to the adversary. Upon receiving $(\text{Verified}, \text{sid}, m, y, \pi, v')$ from the adversary do:
 1. If $v' = v$ for some $(\cdot, v)$ and the entry $T(v, m)$ equals (y, S) with $\pi \in S$, then set $f = 1$.
 2. Else, if $v' = v$ for some recorded pair of the form $(\cdot, v)$, but no entry $T(v, m)$ of the form $(y, \dots, \pi, \dots)$ is recorded, then set $f = \emptyset$.
 3. Else, initialize the table $T(v', \emptyset)$ to empty, and set $f = \emptyset$.
 Output $(\text{Verified}, \text{sid}, m, y, \pi, f)$ to P_i .

Fig. 7. The functionality for verifiable random functions defined in [7].